Diploma project, first submission (probationary period 1)

Development of a Proctoring System for Online Programming Contests

Team: Nurkhan Kuanyshov, Bakytzhan Dzhangazy

Group: SE-2431

Supervisor: Abzal Ermekbayuly Kyzyrkanov, assistant professor, School of Software Engineering

Version: prob-period-1

Date: 24 October 2026

1. Project Proposal
   1.1 Project title
   1.2 Team and supervisor
   1.3 Problem statement
   1.4 Target users
   1.5 Aim
   1.6 Objectives
   1.7 Expected outcomes
   1.8 Project scope
   1.9 Engineering complexity
   1.10 Division of work
   1.11 Plan of the next stage
2. Evidence of Relevance: Interviews with Users
   1. Introduction
   2. Research Objectives
   3. Research Methodology
   4. Participant Overview
   5. Interview Protocols and Transcripts
   6. Consolidated Findings
   7. Verified Facts, Synthetic Findings, and Assumptions
   8. Requirements Traceability Matrix
   9. UX and Product Recommendations
   10. Research Limitations
   11. Conclusion
3. Literature Review
   3.1 Method
   3.2 Review
   3.3 Structured analysis table
   3.4 Gaps and how the project takes them into account
   3.5 References
4. Existing System Comparison
   4.1 Purpose and method
   4.2 Comparison table
   4.3 Findings
   4.4 Limits of this comparison
   4.5 Sources
5. Technology Comparison
   5.1 Method
   5.2 T1 — Server platform
   5.3 T2 — Persistent data storage
   5.4 T3 — Real-time channel from the server to the teacher
   5.5 T4 — Client application framework
   5.6 T5 — In-browser code editor
   5.7 T6 — Integration with DOMjudge
   5.8 T7 — Participant-side integrity control
   5.9 Summary of decisions
   5.10 Sources

6. Requirements Specification
    6.1 Scope and conventions
    6.2 Functional requirements
    6.3 Non-functional requirements
    6.4 Traceability
    6.5 Out of scope for this stage
    6.6 Open points
7. Architecture and Diagrams
    7.1 Architectural principles
    7.2 Diagrams
    7.3 Components and responsibilities
    7.4 Data model
    7.5 API outline (REST, base path `/api/v1`)
    7.6 Design of the critical mechanisms
    7.7 Deployment view
    7.8 Security and privacy
    7.9 Consistency check
    7.10 Known limits and next steps
8. UX / Prototype
    8.1 Status and links
    8.2 Key scenarios
    8.3 Mockups
    8.4 API examples
    8.5 How the prototype will be evaluated
    8.6 Known limits
9. Verification and Evaluation Plan
    9.1 Purpose and status
    9.2 Outcome metrics
    9.3 Testing plan
    9.4 Measurement conditions
    9.5 Baseline for comparison
    9.6 Pilot contest design
    9.7 How the aim will be demonstrated
    9.8 Known limits of the evaluation
10. Project Management and Risks
    10.1 Git repository
    10.2 YouTrack project
    10.3 Allocation of responsibilities
    10.4 Milestones
    10.5 Dependencies
    10.6 Risks
    10.7 Next steps
11. Supervisor Feedback
    11.1 Materials sent for review
    11.2 Feedback and team decisions
    11.3 Summary of how feedback changed the work
Appendix A. Links

A.1 Project resources
A.2 Documents
A.3 Diagram sources and exports
A.4 Mockups

1. Project Proposal

1.1 Project title

Development of a Proctoring System for Online Programming Contests

1.2 Team and supervisor

Member	Group	Role
Nurkhan Kuanyshov	SE-2431	Backend: core system, DOMjudge integration, server-side timer, proctoring events
Bakytzhan Dzhangazy	SE-2431	Frontend and UX/UI: client application, code editor, client-side proctoring, teacher dashboard

Supervisor: Abzal Ermekbayuly Kyzyrkanov, assistant professor, School of Software Engineering.

Project period: October 2026 to May 2027.

1.3 Problem statement

Online contests, lab works and exams in programming have become routine for universities and schools, but control of participant integrity remains a weak point. A participant can copy ready-made code from external sources, switch between windows and messengers, or use hints outside the contest rules. The teacher does not see what happens on the participant's side and lacks reliable data to resolve disputed cases.

Existing solutions cover the problem only partially:

- Judging systems such as DOMjudge verify solutions and maintain the scoreboard, but are oriented to ICPC-style contests and are not designed to control participant behaviour on the participant's side.
- General-purpose proctoring services do not account for the specifics of working with code and are hard to embed into a programming contest workflow.
- A university needs an open, easy-to-deploy tool that can be adapted to its own courses.

> Statements about existing systems are verified in section 4 (Existing system comparison); findings from user discussions are in section 2 (Evidence of relevance). Until then, the claims above are working assumptions.

1.4 Target users

- **Teachers and organizers** who run programming contests, labs and exams online (role *teacher*).

- **Students and school pupils** who participate in contests (role *student*).
- **Developers** who integrate with the system through the REST API.

1.5 Aim

To develop a web-based proctoring system for online programming contests that is integrated with DOMjudge, controls participant integrity on the client side, and gives the teacher real-time monitoring and post-contest reports.

1.6 Objectives

1. Analyze DOMjudge (architecture, API, roles, judgehost), existing contest and proctoring systems, and relevant literature; collect and verify requirements.
2. Specify functional and non-functional requirements with priorities and acceptance criteria.
3. Design the architecture, data model and REST API contract (OpenAPI) for the roles *student* and *teacher*.
4. Prototype the interfaces of both roles and check the prototype with users.
5. Implement the backend: REST API, authentication, DOMjudge integration, server-side timer, proctoring event intake, risk scoring, real-time monitoring and reports.
6. Implement the client: code editor with syntax highlighting, copy/paste blocking, fullscreen and tab-switch control, timer, teacher dashboard, localization (ru, kk, en).
7. Test the system (unit, integration, contract, load, bypass checks) and run a pilot contest with a group of students.
8. Evaluate the results against the target metrics and prepare the explanatory note.

1.7 Expected outcomes

- A working web system: participant client, teacher dashboard and backend integrated with DOMjudge.
- Client-side control mechanisms: syntax highlighting, copy/paste blocking, fullscreen and tab-switch control, timer.
- Documented REST API (OpenAPI), at least 25 endpoints.
- Client application with at least 12 screens for two roles.
- Real-time violation monitoring, incident log and post-contest reports.
- Results of a pilot contest and an explanatory note with technical documentation.

Target metrics

Metric	Target
Client/server timer drift	≤ 1 s
Proctoring event delivery to teacher	≤ 2 s
Concurrent participants	≥ 50
Test coverage of key backend modules	≥ 70%
Share of violations detected in the prepared test scenario	≥ 90%

Metric	Target
Usability score (SUS)	≥ 70

1.8 Project scope

In scope

- Roles *student* and *teacher*, JWT authentication.
- Contests, tasks and participants; submission of solutions to DOMjudge; verdicts and scoreboard.
- Code editor (C++, Python, Java) with syntax highlighting, language selection, templates, draft autosave.
- Client-side proctoring: blocking copy, paste, cut and context menu in the editor; fullscreen control; tab and window switch detection; warnings to the participant; events sent to the server.
- Server-side timer: single time source, start, extension, automatic closing of submissions, synchronization with the client, recovery after connection loss.
- Intake, validation and storage of proctoring events; protection against forged events; risk score per participant.
- Teacher side: contest creation, participant list, real-time monitoring (WebSocket), incident log, scoreboard, reports and results export.
- Localization: Russian, Kazakh, English.
- Containerized deployment and CI.

Out of scope

- Webcam, microphone and screen video recording.
- Replacement or modification of the DOMjudge judging engine.
- Guaranteed prevention of every possible bypass in the browser. The approach is “prevention plus recording” with server-side verification; limitations will be documented.
- Automatic accusations of cheating: the system provides evidence and a risk score, the decision stays with the teacher.

1.9 Engineering complexity

- **Reliability of client-side control:** the browser cannot guarantee blocking of all actions, so a model of prevention plus recording with server-side validation and resistance to bypass is required.
- **Server-side timer:** single time source, client synchronization, resilience to connection loss and page reload, correct closing of submissions.
- **Integration with DOMjudge:** mapping of users, contests and tasks, submitting solutions, retrieving verdicts and scoreboard, handling judging system failures.
- **Security:** separation of roles, protection of the API and proctoring events against forgery, protection of participants’ personal data.
- **Incident analysis:** risk scoring without false accusations, event log for disputed cases, reports.
- **Load and operations:** simultaneous start and submission by dozens of participants, real-time monitoring, container deployment.

1.10 Division of work

The system is split into two contours, each owned by one student. The boundary is fixed by the REST API contract (OpenAPI), so individual contributions can be traced through commit history, tasks and code review. Design of the overall architecture, requirements, integration testing, the pilot contest and the defense are done jointly.

1.11 Plan of the next stage

Period	Work	Result
Nov 2026	Architecture, data model, REST API contract, interface prototype, DOMjudge test stand	Architecture documentation, OpenAPI, prototype, working stand
Dec 2026	MVP-1: roles, authentication, contests and tasks, submission to DOMjudge, code editor	Demo of MVP-1
Jan-Feb 2027	MVP-2: server timer, client proctoring, events and monitoring, teacher dashboard	Demo of MVP-2
Mar 2027	Testing, load tests, bypass checks, pilot contest	Test reports, pilot feedback
Apr-May 2027	Final measurements, explanatory note, pre-defense, defense	Full text, working system

2. Evidence of Relevance: Interviews with Users

Project: Development of a Proctoring System for Online Programming Contests **Document:** `docs/02-evidence-of-relevance.md` **Research period:** 19-23 October 2026

---

1. Introduction

The project extends a DOMjudge-based online programming contest environment with a dedicated interface for students and teachers and with proctoring functionality. Proctoring touches sensitive topics: fairness, privacy and trust. Design decisions made without hearing from the people involved are likely to cause friction, for example warnings that confuse students during a timed contest, or dashboards that show teachers too much and tell them too little.

User interviews are a low-cost way to surface these problems before implementation. In this project they are intended to help identify:

- **Student usability problems:** what causes confusion or lost time in a timed contest.
- **Teacher monitoring needs:** what a teacher must see in order to supervise many participants at once.
- **Concerns about privacy and fairness:** what students consider acceptable monitoring and what they fear.
- **Requirements for event recording and incident review:** what must be recorded, how it should be presented, and who decides what an event means.
- **Opportunities to improve the contest experience:** small changes that remove avoidable stress.

2. Research Objectives

ID	Objective
RO-1	Understand what information students need before and during a contest (rules, time, submission status).
RO-2	Explore how students would react to proctoring notifications and what makes a notification understandable and fair.
RO-3	Identify privacy and false-positive concerns related to tab, fullscreen and clipboard monitoring.
RO-4	Understand how teachers currently supervise contests and what makes remote monitoring difficult.
RO-5	Determine what information teachers need to review a recorded event responsibly, without presuming misconduct.
RO-6	Identify what both roles expect from submission history and post-contest results.

3. Research Methodology

Format. Semi-structured interviews with a prepared question list. The interviewer could ask follow-up questions and skip questions that had already been answered. Questions were phrased neutrally and did not assume that the participant had experienced a particular problem.

Participant profiles (fictional).

- **P01:** a student with some contest experience, focused on clarity and convenience.
- **P02:** a teacher who supervises students and reviews contest results.
- **P03:** a student comfortable with programming, concerned about interruptions, monitoring rules and fairness.

Duration. Approximately 10–20 minutes per interview.

Main topics. Contest experience and difficulties; understanding of rules; coding workspace content; reactions to proctoring warnings; privacy and fairness; teacher supervision practice; event review; post-contest information.

Organizing the findings. Each interview was summarized into short observations. Observations from different participants were grouped into findings (F-xx). For each finding the document records supporting participants, an evidence type, a confidence level, a design implication and open questions. Findings are then linked to provisional requirements in Section 8.

Limitations of simulated responses. Because the responses are written by the researcher, they reflect the researcher's expectations and cannot show how real people think or behave. Section 10 covers this in detail.

Distinction from empirical research. This is a documentation exercise. It demonstrates how real research would be recorded and traced, but it does not replace that research.

4. Participant Overview

Participant ID	Role	Relevant Experience	Interview Date	Duration	Main Focus
P01	Student / Contest Participant	Third-year student; has taken part in a handful of online contests, mostly university-organized	12 Oct 2026	14 min	Clarity, convenience, avoiding confusion under time pressure
P02	Teacher / Contest Organizer	Lecturer in algorithms; has organized in-class and online contests and reviewed results	14 Oct 2026	18 min	Monitoring efficiency, review of evidence, avoiding unfair accusations

Participant ID	Role	Relevant Experience	Interview Date	Duration	Main Focus
P03	Student / Contest Participant	Second-year student; confident programmer; has had connection problems during an online exam	16 Oct 2026	16 min	Privacy, false positives, technical interruptions, clarity of rules

---

5. Interview Protocols and Transcripts

Interview P01

Field	Value
Interview ID	INT-01
Participant ID	P01
Participant role	Student / Contest Participant
Date	12 October 2026
Duration	14 minutes
Interview objective	Learn what a student needs to see and understand before and during a timed contest, and how they would respond to proctoring notifications (RO-1, RO-2, RO-6).

Interview questions and responses

Q1. Can you describe your experience with online programming contests? > I've done maybe five or six. Mostly the university ones, plus a couple of open contests where I just wanted to practice. I'm not fast, but I usually solve two or three problems.

Q2. What usually causes the most difficulty during a timed contest? > Honestly, losing track of things. I lose track of time, or I forget which problems I've already submitted. Once I thought I'd sent problem B and I hadn't. I only noticed with ten minutes left.

Q3. How do you understand the contest rules before starting? > Usually there's a PDF or a message from the teacher. I skim it. Sometimes I miss things, like whether you can use the internet for documentation. I think a short rules page right before the start would help, as long as it's actually short.

Q4. What information do you need to see in the coding workspace? > The problem statement and the editor side by side. The timer, always. And the status of my last submission, because I keep wondering if it's still being judged or if it's stuck. I'd also like to see which problems I've already solved without opening each one.

Q5. How would you react if the system detected that you switched browser tabs? > Depends how it tells me. If a big red box appears I'd panic a bit, even if I only checked something innocent. If it says plainly "we recorded that you left the contest tab", then I'd just go back.

Q6. What would make a proctoring warning understandable and fair? > Say what happened and when. Say what I should do now. And don’t make it sound like I’m being accused. “Recorded” is OK, “cheating detected” is not.

Q7. What concerns would you have about clipboard or fullscreen monitoring? > Fullscreen I don’t mind, it’s like an exam. Clipboard is less clear to me. I copy my own code sometimes, for example into a scratch file. I don’t know if that counts. I wouldn’t be against it, I’d just want to know the rule in advance.

Q8. What should happen if a warning is triggered accidentally? > I guess I’d want a quick way to see what was recorded. Maybe a note that says “if this was a mistake, tell your teacher”. I wouldn’t want to stop the contest to write an explanation.

Q9. How useful would a submission history be? > Very. I’d look at it a lot. Time, verdict, which problem. Also which language, because I switched from Python to C++ once and lost track.

Q10. Is there anything about online contests you would change? > The waiting. Sometimes the verdict takes ages and there’s no sign of progress. Even a “queued / judging” label would be better than a blank screen.

Interviewer notes - Participant spoke quickly and practically. Most answers came from one specific past incident (the unsent problem B). - Showed no strong objection to monitoring; the concern was tone and clarity, not the monitoring itself. - Clipboard rule was the only area of real uncertainty.

Preliminary observations - Rules should be short and shown immediately before the contest. - Timer and submission status are the key persistent elements. - Notification wording matters more to this participant than the technical detail of what is detected. - Needs a way to see per-problem solved status.

Interview P02

Field	Value
Interview ID	INT-02
Participant ID	P02
Participant role	Teacher / Contest Organizer
Date	14 October 2026
Duration	18 minutes
Interview objective	Understand how a teacher supervises contests today, what they need from a monitoring dashboard, and how they would review events responsibly (RO-4, RO-5, RO-6).

Interview questions and responses

Q1. How do you currently organize or supervise programming contests? > In class, I just walk around. For online ones it’s a video call with cameras optional, plus the judging system’s scoreboard. I set up the problems in the judge and share a link. Honestly the setup takes longer than the contest.

Q2. What is difficult about monitoring multiple participants remotely? > I can’t see what anyone is doing. With thirty students I can’t watch thirty screens anyway. Mostly I notice problems after the fact,

when two solutions look suspiciously alike. By then it’s hard to say anything for sure.

Q3. What information would you want on a monitoring dashboard? > A list of students, who is online, what they’ve solved, when they last did something. If someone’s offline for ten minutes I want to know, because they may have a real problem. I don’t need pretty charts. A table I can sort is enough.

Q4. How should recorded events be presented? > Short and in order. “14:32, left fullscreen, back after 8 seconds.” Not a score. If it says “risk 82%” I’d not know what to do with that, and a colleague might treat it as proof.

Q5. How do you distinguish a suspicious event from an ordinary technical issue? > Context. One tab switch means nothing; people check the clock. A student who switches many times right before a correct submission, that deserves a conversation. But even then it’s a conversation, not a verdict. Also, connection drops: if the log shows a disconnect around the same time, that explains a lot.

Q6. What information should be available when reviewing a participant? > The event list, their submissions with times, and what they were working on. And whether they were warned. If they were warned three times and kept going, that’s different from never being warned.

Q7. Would review notes or an event timeline be useful? Why? > The timeline, yes. Notes, I think yes, but only if they’re quick. I’d write “spoke with student, was checking the contest clock” and I’d want that to stay attached. Otherwise I’d forget in a week. I’m less sure whether the student should see my notes.

Q8. What information is needed after a contest ends? > Results with who solved what and the penalty times, obviously. And a list of the flagged participants, so I can review those without hunting through everyone. I’d export the results for grading; not sure how much of the event data I’d need in the export.

Q9. What should the system avoid doing automatically? > Disqualifying anyone, or blocking submissions because of an event. Also sending anything to the student’s department. It can warn the student, fine, but any decision is mine.

Q10. What would make this tool useful in practice? > If it saves me time. If I open it and in thirty seconds see who needs attention, that’s useful. If I have to click through every student, I’ll go back to the video call. I’m also unsure whether I’d want live alerts. It might be distracting during the contest, I’d probably prefer to glance at it when I choose.

Interviewer notes - Participant was cautious about anything resembling automated judgment and returned to the point several times. - Was undecided on live alerts versus periodic review, and said so openly. - Mentioned colleagues treating numeric scores as proof, as a general worry, with no specific case given.

Preliminary observations - Dashboard should be a sortable table with minimal decoration. - Events should be shown as a plain chronological list, with no numeric suspicion score. - Disconnection information is needed to interpret events. - Review notes are wanted, but who may read them is unresolved. - Live alert preference is unresolved.

Interview P03

Field	Value
Interview ID	INT-03
Participant ID	P03
Participant role	Student / Contest Participant
Date	16 October 2026
Duration	16 minutes
Interview objective	Explore student concerns about privacy, false positives and technical interruptions, and what explanation of monitoring they would consider adequate (RO-2, RO-3, RO-6).

Interview questions and responses

Q1. Can you describe your experience with online programming contests? > A few, mostly through the faculty. I like them, they’re a good way to practice. I also had an online exam last year where my connection dropped, so I’m a bit sensitive about this topic.

Q2. What usually causes the most difficulty during a timed contest? > Besides the problems themselves? Technical stuff. Connection, the page freezing, not knowing whether my submission went through. When the clock is running you can’t afford to wonder.

Q3. How do you understand the contest rules before starting? > Mostly I don’t read them properly, I’ll admit. If it’s a long text I skip. But with monitoring, I’d actually read it, because I want to know what exactly is recorded. Is it only that I left the tab, or the contents of the clipboard, or what? That difference matters to me.

Q4. What information do you need to see in the coding workspace? > Timer, statement, editor, results. And a way to know I’m connected. If the connection drops I want it to say so clearly, so I don’t keep typing into nothing. Also, saved code. I’d want to know my code is not lost if the page reloads.

Q5. How would you react if the system detected that you switched browser tabs? > It depends on whether it was me. If I clicked away on purpose, fine. But sometimes a notification pops up on my laptop and the window loses focus without me doing anything. That is what worries me. I’d feel annoyed if I got a warning for it.

Q6. What would make a proctoring warning understandable and fair? > Explain exactly what triggered it. And tell me what happens next: does the teacher see it, does it affect my score? If I don’t know the consequences I’ll assume the worst. Also don’t count the same thing five times because the window flickered.

Q7. What concerns would you have about clipboard or fullscreen monitoring? > Clipboard more than fullscreen. I’m fine with “you pasted something”. I’m not fine with anything that reads what’s inside my clipboard, because it might have personal stuff in it. Also, the problem statement has sample inputs; I’d copy those to test. That must not be treated as suspicious.

Q8. What should happen if a warning is triggered accidentally? > I should be able to tell someone. A button like “this was accidental” would be good, even if it only adds a note for the teacher. And the

teacher should see that note next to the event.

Q9. How useful would a submission history be? > Useful, but mainly for reassurance. To prove it was submitted and when. If something goes wrong, that history is my evidence.

Q10. Is there anything about online contests you would change? > Tell us more about what's happening. Rules, monitoring, what's recorded and who sees it. I don't think students mind rules if they're stated honestly.

Interviewer notes - Participant was calm but pointed about privacy, and kept returning to what is stored and who sees it. - Raised an interesting tension: copying sample input is legitimate but looks like clipboard activity. - Previous connection-loss experience clearly shaped the answers.

Preliminary observations - The explanation of what is recorded (and what is not) should be stated up front, including that clipboard contents are not read. - Focus loss can be unintentional, so warnings need wording and possibly a way to flag an accident. - Connection state and autosave are trust-critical. - Copying sample data from the statement needs a defined policy.

---

6. Consolidated Findings

All findings are **synthetic and unvalidated**. "Confidence" refers only to how consistently the finding appears across the three fictional interviews, not to real-world certainty. No quantitative claims are made.

F-01 — Clear contest rules before starting

- **Description:** Participants want a short, readable statement of the rules, including what is monitored, before the contest begins.
- **Supporting participants:** P01, P03
- **Relevant response (paraphrase):** P01 wants a short rules page just before the start. P03 said she would actually read the monitoring rules because she wants to know what is recorded.
- **Evidence type:** Synthetic interview statement
- **Confidence:** Medium (two participants, same direction)
- **Design implication:** Show a concise rules and monitoring summary on the contest details screen, with an acknowledgment step before entering the workspace.
- **Unresolved:** How long is "short"? Do students read it, or just tick the box?

F-02 — Visible timer and submission status

- **Description:** A persistent timer and a clear status for the latest submission reduce uncertainty under time pressure.
- **Supporting participants:** P01, P03
- **Relevant response (paraphrase):** P01 wants the timer always visible and a "queued / judging" label instead of a blank screen. P03 does not want to wonder whether a submission went through.
- **Evidence type:** Synthetic interview statement
- **Confidence:** Medium
- **Design implication:** Keep the timer in the workspace header; show per-submission states (queued, judging, verdict).
- **Unresolved:** Does a prominent timer increase stress for some students?

F-03 — Transparent proctoring notifications

- **Description:** Notifications should say what was recorded, when, and what the participant should do, in neutral language.
- **Supporting participants:** P01, P03
- **Relevant response (paraphrase):** P01: “recorded” is acceptable, “cheating detected” is not. P03 wants to know exactly what triggered a warning and what happens next.
- **Evidence type:** Synthetic interview statement
- **Confidence:** Medium
- **Design implication:** Warnings with event name, timestamp, a statement that the event is recorded for review, and a recovery action. Severity proportional to the event.
- **Unresolved:** Do students prefer detailed or minimal warnings?

F-04 — Human review of recorded events

- **Description:** Recorded events should be interpreted by a teacher, not turned into automatic outcomes.
- **Supporting participants:** P02, P03
- **Relevant response (paraphrase):** P02 said any decision is the teacher’s. P03 wants to know whether warnings affect the score.
- **Evidence type:** Synthetic interview statement
- **Confidence:** Medium
- **Design implication:** No automatic disqualification or blocking; the review workflow ends with a teacher-set status.
- **Unresolved:** Which review statuses are needed?

F-05 — Participant activity timeline

- **Description:** Teachers want a chronological, plain list of events per participant, alongside submissions and warnings.
- **Supporting participants:** P02
- **Relevant response (paraphrase):** P02 wants entries such as time, event and duration away, and wants to see whether the student was warned.
- **Evidence type:** Synthetic interview statement (single participant)
- **Confidence:** Low (one participant)
- **Design implication:** A participant detail view combining events, submissions and warnings on one timeline.
- **Unresolved:** Is the same view needed by other teachers? How much history is useful?

F-06 — Protection against unfair automated conclusions

- **Description:** Both teachers and students are wary of scores or labels that read as proof of misconduct.
- **Supporting participants:** P02, P03
- **Relevant response (paraphrase):** P02 would not know what to do with a risk percentage and worries colleagues might treat it as proof. P03 wants a way to flag accidental events.
- **Evidence type:** Synthetic interview statement
- **Confidence:** Medium

- **Design implication:** Avoid suspicion scores; use neutral labels such as “needs review”; allow student-added context for an event.
- **Unresolved:** Is any attention indicator acceptable, and in what form?

F-07 — Useful teacher monitoring dashboard

- **Description:** A compact, sortable table showing who is online, progress, recent activity and event counts is valued over decorative visuals.
- **Supporting participants:** P02
- **Relevant response (paraphrase):** P02: a table she can sort is enough; she wants to see who needs attention in thirty seconds. Disconnections matter.
- **Evidence type:** Synthetic interview statement (single participant)
- **Confidence:** Low
- **Design implication:** Participants table with connection status, last activity, event count and filters, rather than charts.
- **Unresolved:** Live alerts or periodic checking? P02 was undecided.

F-08 — Accessible submission history

- **Description:** Both students and the teacher want to see submissions with time, problem, language and verdict.
- **Supporting participants:** P01, P02, P03
- **Relevant response (paraphrase):** P01 uses history to track what has been sent and in which language. P03 sees it as proof of submission. P02 uses submissions in review.
- **Evidence type:** Synthetic interview statement
- **Confidence:** Medium
- **Design implication:** History available inside the workspace and in teacher review, with the same fields.
- **Unresolved:** How much detail (e.g. test case results) is appropriate during a contest?

F-09 — Connection state and recovery

- **Description:** Connection drops are a real source of stress and can resemble suspicious activity in the event log.
- **Supporting participants:** P03, P02
- **Relevant response (paraphrase):** P03 wants a clear “connection lost” indicator and confidence that code is saved. P02 said a disconnect near an event explains a lot.
- **Evidence type:** Synthetic interview statement
- **Confidence:** Medium
- **Design implication:** Connection indicator for students; connection events shown in teacher timeline; autosave visible.
- **Unresolved:** How should the system treat events recorded around a disconnect?

F-10 — Clipboard policy needs to be explicit

- **Description:** Participants differ on clipboard monitoring. One is unsure whether copying own code counts; another accepts “paste detected” but not any reading of clipboard content, and wants copying sample input to be allowed.
- **Supporting participants:** P01, P03

- **Relevant response (paraphrase):** P01 would accept the rule if stated in advance. P03 distinguishes recording an action from reading its contents.
- **Evidence type:** Synthetic interview statement (divergent views)
- **Confidence:** Low (views differ)
- **Design implication:** State what clipboard events are recorded (action only, not content). Decide how copying from the statement is treated.
- **Unresolved:** Is distinguishing in-page sample copying technically and ethically sufficient?

---

7. Verified Facts, Synthetic Findings, and Assumptions

A. Verified Project Facts

> **Source note:** These facts come from the project brief and design task supplied by the project author. They should be cross-checked against the repository's requirements specification and architecture documents when finalizing this document, and any difference resolved in favor of the repository.

- The project title is *Development of a Proctoring System for Online Programming Contests*.
- The system is a web-based platform built on top of DOMjudge, which handles contest administration, problem statements, submission processing, automated judging, verdicts and standings.
- The system adds a dedicated user experience for students and teachers, and proctoring functionality.
- There are two primary roles: Student / Contest Participant and Teacher / Contest Organizer.
- Students can log in, view contests, join a contest, read problems, write code with syntax highlighting, select a language, submit, view history and verdicts, see remaining time and receive warnings.
- Teachers can log in, create and configure contests, view participants, monitor them, see proctoring events in real time, identify participants who may need attention, and review incidents and results.
- Events to be detected and recorded include tab or window switching, leaving fullscreen, and copy or paste attempts, plus other events defined by the project requirements.
- The approach is prevention plus detection. Browser-based restrictions are not presented as foolproof; events are recorded and reviewed.
- The monitoring interface should support informed review and must not present a suspicion score as proof of cheating.
- Out of scope: AI-based face recognition, webcam recording and biometric analysis.
- The planned frontend stack is React, TypeScript, Vite, Tailwind CSS, Monaco Editor, TanStack Query, WebSocket and i18next.

B. Synthetic Interview Findings

These are synthetic and unvalidated. Patterns across INT-01 to INT-03:

- Students (P01, P03) wanted short, clear rules and plain-language, non-accusatory warnings that say what happened and what to do next.

- Students wanted the timer, submission status, autosave and connection state to be clearly visible.
- The teacher (P02) wanted a compact table and plain chronological events, with decisions left to a human.
- Teacher and student both distrusted automated labels or scores.
- Clipboard monitoring was the least settled topic.
- Submission history was valued by all three, for different reasons.

C. Assumptions Requiring Validation

#	Assumption	How it could be validated
A-1	Students prefer a short, specific warning over a detailed one.	A/B test of warning wording in a usability session; post-task questions.
A-2	Teachers prefer checking a dashboard on demand to receiving live alerts.	Interview more teachers; observe a pilot contest; compare alert and no-alert conditions.
A-3	Tab switches, fullscreen exits and clipboard actions are the most useful event types for review.	Review recorded events from a pilot contest with teachers and ask which they actually used.
A-4	A timeline view gives enough detail for incident review.	Task-based testing: ask teachers to review a staged incident and note what they miss.
A-5	Students accept recording of clipboard actions if the rule is stated in advance.	Survey and interviews with a larger sample; ask separately about actions versus contents.
A-6	Students should be able to add context to an event.	Test a prototype of the “add context” action; ask teachers whether the notes affect their judgement.
A-7	Teachers need notes that are private to teachers.	Ask several teachers and students about note visibility.
A-8	A prominent timer does not increase stress to a harmful degree.	Usability testing with a timed task and a short stress questionnaire.

---

8. Requirements Traceability Matrix

The IDs are **provisional**. If the repository contains a requirements specification, its IDs should replace these and the matrix should be updated. Validation status labels: `Supported by project scope`, `Suggested by synthetic research`, `Requires real-user validation`.

Finding ID	Evidence Type	User Need	Requirement ID	Proposed Requirement	Priority	Validation Status
—	Project scope	Access the platform with the correct role	FR-01 — Authentication	Users log in and are directed to the student or teacher interface.	High	Supported by project scope

Finding ID	Evidence Type	User Need	Requirement ID	Proposed Requirement	Priority	Validation Status
—	Project scope	Find contests by state	FR-02 — Contest List	Show available, ongoing, upcoming and completed contests with start time and duration.	High	Supported by project scope
F-01	Synthetic interview + project scope	Know the rules and monitoring before starting	FR-03 — Contest Details and Rules	Show rules and monitored events on the details screen; require acknowledgment before entering.	High	Supported by project scope; acknowledgment step suggested by synthetic research
F-02, F-09	Synthetic interview + project scope	Stay oriented under time pressure	FR-04 — Programming Workspace	Provide statement and editor, language selection, persistent timer, autosave indicator and connection indicator.	High	Supported by project scope; indicators suggested by synthetic research
F-02	Synthetic interview + project scope	Know whether a submission was received	FR-05 — Code Execution and Submission	Run and submit code; show queued, judging and final verdict states.	High	Supported by project scope; intermediate states suggested by synthetic research
F-08	Synthetic interview + project scope	Track what was submitted	FR-06 — Submission History and Results	List submissions with problem, language, time, verdict, runtime and memory.	High	Supported by project scope
F-03, F-06	Synthetic interview + project scope	Understand a warning without feeling accused	FR-07 — Proctoring Notifications	Show event, timestamp, “recorded for review” wording and a recovery action; severity proportional to the event.	High	Supported by project scope; wording suggested by synthetic research
F-06, F-10	Synthetic interview	Add context to an accidental event	FR-07a (proposed extension of FR-07)	Allow the participant to add a short note to a recorded event.	Medium	Requires real-user validation

Finding ID	Evidence Type	User Need	Requirement ID	Proposed Requirement	Priority	Validation Status
F-09, F-10	Synthetic interview + project scope	Reliable event data	FR-08 — Proctoring Event Recording	Record event type, time and duration; include connection state events; record clipboard actions without clipboard contents.	High	Supported by project scope; content exclusion and connection events suggested by synthetic research
F-07	Synthetic interview + project scope	See who needs attention quickly	FR-09 — Teacher Monitoring Dashboard	Sortable participants table with progress, connection status, last activity, event count and filters.	High	Supported by project scope; layout details require real-user validation
F-05, F-04	Synthetic interview + project scope	Review one participant with context	FR-10 — Participant Event Review	Timeline of events, warnings and submissions; review notes and a status set by the teacher.	High	Supported by project scope; timeline suggested by synthetic research
F-08	Synthetic interview + project scope	Grade and conclude	FR-11 — Contest Results	Scoreboard with solved problems and penalty time; list of participants flagged for review.	Medium	Supported by project scope; flagged list suggested by synthetic research
F-01, F-02, F-03	Synthetic interview	Clear interface under pressure	NFR-01 — Usability and Clarity	Plain language, consistent status colors, minimal distraction in the workspace.	High	Suggested by synthetic research; requires real-user validation
F-04, F-06	Synthetic interview + project scope	Fair handling of events	NFR-02 — Fairness and Human Review	No automatic penalties or numeric suspicion scores; events are labelled as indicators for human review.	High	Supported by project scope
F-03, F-10	Synthetic interview	Know what is stored and who sees it	NFR-03 — Privacy and Transparency	Publish what is recorded, what is not, and who can access it.	High	Suggested by synthetic research; requires real-user validation

Finding ID	Evidence Type	User Need	Requirement ID	Proposed Requirement	Priority	Validation Status
F-07	Assumption A-2	Choice of live alerts or periodic review	FR-09 (open item)	Decide whether real-time alerts are needed in addition to the table.	Low	Requires real-user validation

---

9. UX and Product Recommendations

Source labels: **[PS]** project scope, **[SF]** synthetic finding, **[AV]** assumption requiring validation.

Student interface

Recommendation	Source
Show a rules and monitoring summary on the contest details screen; require acknowledgment before entering the workspace.	[PS] rules and monitoring; [SF] F-01 for the acknowledgment step
Keep a countdown timer visible in the workspace header.	[PS] remaining time; [SF] F-02
Show submission states (queued, judging, verdict) and a per-problem solved indicator.	[SF] F-02, F-08
Use proportionate, neutral warnings: event, time, “recorded for review”, what to do.	[PS] warnings; [SF] F-03
Include a plain explanation of monitored events, including that clipboard contents are not read.	[PS] event types; [SF] F-10; the content-exclusion rule requires [AV] A-5
Make clear that a recorded event is not an accusation or a decision.	[PS] human review; [SF] F-04, F-06
Offer recovery guidance (return to fullscreen, an optional “add context” note).	[SF] F-06; [AV] A-6
Show autosave and connection indicators.	[SF] F-09

Teacher interface

Recommendation	Source
Concise participants table with progress, last submission, connection, event count and last activity.	[PS] participant monitoring; [SF] F-07
Severity labels that describe the event, not the person.	[PS] no suspicion score as proof; [SF] F-06
Filtering and sorting (e.g. participants with incidents, offline participants).	[PS] identify participants needing attention; [SF] F-07
Participant-specific timeline combining events, warnings and submissions.	[PS] incident review; [SF] F-05
Review notes attached to events; decision on note visibility pending.	[PS] review; [SF] F-05; visibility is [AV] A-7

Recommendation	Source
Access to submissions and contest info from the review screen.	[PS] results and submissions; [SF] F-08
A neutral review process: statuses such as “not reviewed”, “reviewed – no concern”, “follow up”.	[PS] human review; [SF] F-04; exact statuses are [AV]
Optional live alerts.	[AV] A-2

---

10. Research Limitations

- The interviews are fictional and written by the researcher.
- No real participants were recruited, and no real conversations or observations took place.
- The responses reflect the researcher’s expectations and may emphasize issues the researcher already considered important.
- With three fictional participants, no claim about how common any view is, or about statistical significance, is possible.
- Real students and teachers may raise different priorities, such as concerns not anticipated here, or may disagree with these.
- Only one teacher perspective is represented, and there are no administrators or other staff.

Why the document is still useful. It works as a prototype for research documentation: it fixes a protocol, a findings format and a traceability method that can be applied unchanged to real data. It also records design hypotheses (Section 7C) and links them to requirements, so that real research can be targeted at the points of highest uncertainty.

11. Conclusion

The simulated interviews suggest that the system should concentrate on clarity, transparency, usability, teacher oversight and fair handling of events. For students, this means short, honest rules, visible time and submission status, and warnings that explain without accusing. For teachers, it means a compact monitoring view and a timeline that supports judgement, with the decision left to a person. These are design directions and hypotheses. The synthetic interviews do not prove the system’s relevance, which needs to be established through real research.

3. Literature Review

3.1 Method

Sources were collected in October 2026 through web search of scholarly venues (arXiv, ACM Digital Library, USENIX, PeerJ, Frontiers, CAIS, ScitePress) and the websites of the tools discussed. Search topics:

1. proctoring and cheating detection in online programming exams;
2. academic integrity in programming contests and courses;
3. student and faculty perceptions of remote proctoring (privacy, anxiety, false accusations);
4. plagiarism detection for source code, including LLM-generated code;
5. DOMjudge and automated judging in teaching and contests;
6. browser-based integrity signals (tab switching, copy/paste).

Selection criteria: relevance to online programming assessment or contests; preference for sources from 2021 to 2026; foundational or industrial sources only where needed to describe existing tools. Result: 17 sources, of which 10 are peer-reviewed publications, 5 are preprints and 2 are tool documentation or industry sources.

3.2 Review

3.2.1 Proctoring of online programming exams

Heavy webcam- and vision-based proctoring dominates the literature. A framework for multimodal proctoring [13] combines computer vision for faces, objects and hand-object interaction, and states openly that human proctoring does not scale while many automated solutions are intrusive. Work on abnormal behavior detection in online exams [12] follows the same direction.

For programming exams specifically, Ortin et al. [1] propose a machine-learning assistant that analyzes sequences of screenshots with a CNN and RNN to help instructors detect fraud in real time. Their paper also recalls earlier systems with very high accuracy and recall but extremely low precision, which illustrates that false positives are the central difficulty of automated detection.

Studies of behavior-based signals are fewer. Akçapınar [11] analyzed exam logs of 52 students with clustering, using counts of text selections, right-clicks and page focus losses as indicators of possible AI-assisted cheating. The study is preliminary, but it shows that cheap browser events carry information about suspicious behavior without cameras.

3.2.2 Cheating and integrity in programming

Goldberg [6], writing from the experience of teaching online programming courses, argues that standard anti-cheating measures are insufficient and that cheating is more widespread than many assume, including through paid help. He recommends redesigning course assessment rather than relying on a single tool.

Industry practice documents the same logic. HackerRank [17] describes tab proctoring that detects tab and window focus changes and copy-paste activity, and also lists what such tools miss: other devices, mobile hotspots and browser extensions that hide switching. This supports the position that browser events are useful as evidence, but not as proof.

Dickey [14], drawing on seven semesters of teaching a competitive programming course, notes that tasks with unique known solutions make copying hard to distinguish, and advocates combining detection algorithms, mastery-based learning and authentic assessment.

3.2.3 Plagiarism detection and generative AI

Similarity detectors compare submitted code. Gacek et al. [9] evaluate MOSS, JPlag, DOLOS and copydetect and report a trade-off between sensitivity and specificity: some tools resist false positives but are vulnerable to obfuscation, others the opposite.

Generative models make this harder. Biderman and Raff [7] showed that a language model could complete introductory programming assignments without triggering MOSS, and that the generated code varied in structure with no obvious signature. Maisch et al. [8] asked students to plagiarize on purpose and document how: few succeeded in evading detection, but the strategies were varied and often LLM-based.

Taken together, post-hoc code similarity analysis alone is not enough for a contest where participants may use an LLM. Behavioral logs from the participant’s side are a complementary source of evidence.

3.2.4 Student and faculty perceptions of proctoring

Alessio and Messinger [3] surveyed 150 faculty and 78 students: most students and about half of faculty saw proctoring as an invasion of privacy, and students reported stress and anxiety. Balash et al. [2] analyzed browser extensions of proctoring services and surveyed students: students accept some loss of privacy but are concerned about sharing personal data, and institutional power dynamics may reduce open opposition. Meulmeester et al. [4] surveyed 597 medical students; the main worries were wrongful invalidation of an exam due to connection, noise or webcam problems, and the fear of being flagged incorrectly. Bergmans et al. [5] found that students using a commercial tool considered it easy to set up and believed that proctoring itself deters cheating.

Common conclusions: false flags, technical failures and privacy are the main concerns, while deterrence is perceived as the main benefit.

3.2.5 DOMjudge and automated judging

DOMjudge was designed for ICPC-style contests, has been in use for over two decades, handles sandboxing of untrusted code and is language-independent by design [15]. It is also used in teaching: the Hong Kong Polytechnic University case study [10] used DOMjudge to judge a Python contest for an introductory course. DOMjudge exposes a REST API and has jury, team and admin roles [16], which makes integration possible without modifying the judging engine. The literature on DOMjudge focuses on grading and learning outcomes, not on behavioral integrity control.

3.3 Structured analysis table

#	Source	Type	Year	Topic	Method / data	Main finding	Limitation for our project
1	Ortin et al., PeerJ CS	Journal	2025	ML detection in online programming exams	CNN+RNN on screenshot sequences	ML can assist instructors in real-time detection	Needs screen capture; false-positive risk; heavy
2	Balash et al., SOUPS	Conf.	2021	Student privacy perceptions	Review analysis of extensions, survey n=102	Privacy concerns, trust in institution shapes acceptance	General exams, not programming
3	Alessio, Messinger, Front. Educ.	Journal	2021	Faculty/student perceptions	Survey, 150 faculty, 78 students	Privacy invasion and stress are widespread concerns	Pandemic context, not contests
4	Meulmeester et al.	Journal	2021	Student perspectives	Survey, 597 students	Fear of wrongful invalidation and false flags	Medical progress tests
5	Bergmans et al.	Conf.	2021	Efficacy of Proctorio	Student study	Easy to use; seen as deterrent	One commercial tool
6	Goldberg, CAIS	Journal	2021	Integrity in online coding courses	Experience-based analysis	Existing measures insufficient; redesign assessment	Essay, no quantitative data
7	Biderman, Raff, CIKM	Conf.	2022	LLM vs. plagiarism detectors	Experiment with GPT-J and MOSS	LLM code evades MOSS	Older model
8	Maisch et al.	Conf.	2026	How students plagiarize	Empirical challenge study	Varied manual and LLM strategies	Assignments, not live contests
9	Gacek et al., CSEDU	Conf.	2026	Plagiarism detectors comparison	Evaluation of 4 tools	Sensitivity/specificity trade-offs	Post-hoc analysis only
10	Yuen, Liu, Leong, Comput. Appl. Eng. Educ.	Journal	2023	DOMjudge in a Python contest	Case study at PolyU	DOMjudge suits contest-style teaching	No integrity control
11	Akçapınar, arXiv:2510.18881	Preprint	2025	AI-assisted cheating from behavior	Clustering, 52 students	Focus loss, selections, right-clicks as indicators	Preliminary, small sample
12	Ngo et al., arXiv:2402.12179	Preprint	2024	Abnormal behavior detection	Monitoring system	Describes detection approach	Not programming-specific
13	AutoOEP, arXiv:2509.10887	Preprint	2025	Multimodal proctoring	Computer vision	Combines face, object, hand tracking	Intrusive, webcam required

#	Source	Type	Year	Topic	Method / data	Main finding	Limitation for our project
14	Dickey, arXiv:2505.08244	Preprint	2025	Plagiarism in competitive programming	Literature-based analysis	Multi-faceted approach needed	Review, not a tool
15	Meggendorfer, DOMtutor, arXiv:2608.16899	Preprint	2026	DOMjudge as autograder	Description and use cases	DOMjudge mature, versatile, sandboxed	Teaching focus
16	DOMjudge	Tool site	n/a	Judging system	Documentation	REST API, roles, judgehosts	n/a
17	HackerRank, Tab-Proctoring	Industry	n.d. (accessed Oct 2026)	Browser proctoring	Vendor article	What tab proctoring catches and misses	Vendor source

3.4 Gaps and how the project takes them into account

Gap in the literature	How the project responds
Most proctoring relies on webcam/CV, which students find intrusive and which raises false flags [2, 3, 4, 13]	No webcam or video. Browser-level events only (tab switch, fullscreen exit, copy/paste)
High recall with very low precision in automated detection [1]	The system outputs a risk score and evidence log; the teacher decides. No automatic accusation
Few studies use cheap behavioral signals for programming exams [11]	Event logging on the client with server-side validation, evaluated in a pilot contest
Browser signals can be bypassed [17]	“Prevention plus recording”; limitations documented and bypass tests planned
LLMs defeat code-similarity detection [7, 8, 9]	Behavioral log complements post-hoc analysis
DOMjudge literature focuses on grading, not integrity [10, 15]	Integrate with DOMjudge through its REST API and add proctoring on top

3.5 References

[1] F. Ortin, A. Gago, J. Quiroga, M. Garcia. A machine learning assistant for detecting fraudulent activities in synchronous online programming exams. PeerJ Computer Science, 11:e3159, 2025. https://doi.org/10.7717/peerj-cs.3159

[2] D. G. Balash, D. Kim, D. Shaibekova, R. A. Fainchtein, M. Sherr, A. J. Aviv. Examining the Examiners: Students’ Privacy and Security Perceptions of Online Proctoring Services. SOUPS 2021, pp. 633-652. https://www.usenix.org/conference/soups2021/presentation/balash

[3] H. M. Alessio, J. D. Messinger. Faculty and Student Perceptions of Academic Integrity in Technology-Assisted Learning and Testing. Frontiers in Education, vol. 6, 2021. https://doi.org/10.3389/feduc.2021.629220

[4] F. L. Meulmeester, E. A. Dubois, C. Krommenhoek-van Es, P. G. M. de Jong, A. M. J. Langers. Medical students’ perspectives on online proctoring during remote digital progress test. 2021.

https://doi.org/10.1007/s40670-021-01420-w

[5] L. Bergmans, N. Bouali, M. Luttikhuis, A. Rensink. On the Efficacy of Online Proctoring using Proctorio. 2021. https://doi.org/10.5220/0010399602790290 [6] D. M. Goldberg. Programming in a Pandemic: Attaining Academic Integrity in Online Coding Courses. Communications of the AIS, 48, pp. 47-54, 2021. https://doi.org/10.17705/1CAIS.04807

[7] S. Biderman, E. Raff. Fooling MOSS Detection with Pretrained Language Models. Proc. CIKM 2022, pp. 2933-2943. https://doi.org/10.48550/arXiv.2201.07406

[8] R. Maisch, L. Schmid, R. Glassey, D. Fuchss, N. Niehues, H. Liu, A. Koziolek. Criminal Minds: How First-Year CS Students Plagiarize Code. 2026. https://doi.org/10.1145/3803437.3805789

[9] P. Gacek, B. Gdowski, K. Szymanski, W. Zmuda. Comparative analysis of non-commercial plagiarism detectors for computer science education. Proc. CSEDU 2026. https://doi.org/10.5220/0014836500004021

[10] K. K. F. Yuen, D. Y. W. Liu, H. V. Leong. Competitive programming in computational thinking and problem solving education. Computer Applications in Engineering Education, 31(4), pp. 850-866, 2023. https://doi.org/10.1002/cae.22610 (open-access manuscript: https://ira.lib.polyu.edu.hk/handle/10397/97220)

[11] G. Akçapınar. Detecting AI-Assisted Cheating in Online Exams through Behavior Analytics. arXiv:2510.18881, 2025. https://arxiv.org/abs/2510.18881

[12] D. A. Ngo, T. D. Nguyen, T. L. C. Dang, H. H. Le, T. B. Ho, V. T. K. Nguyen, T. T. H. Nguyen. Examining Monitoring System: Detecting Abnormal Behavior In Online Examinations. arXiv:2402.12179, 2024. https://arxiv.org/abs/2402.12179

[13] AutoOEP: A Multi-modal Framework for Online Exam Proctoring. arXiv:2509.10887, 2025. https://arxiv.org/abs/2509.10887

[14] E. Dickey. The Failure of Plagiarism Detection in Competitive Programming. arXiv:2505.08244, 2025. https://arxiv.org/abs/2505.08244

[15] T. Meggendorfer. DOMtutor: Automated Autograding for Logic in Computer Science. arXiv:2608.16899, 2026. https://arxiv.org/abs/2608.16899

[16] DOMjudge: Programming contest jury system. https://www.domjudge.org/ .

[17] HackerRank. Tab-Proctoring: What It Catches (and What It Misses). Undated web article, accessed October 2026. https://www.hackerrank.com/writing/tab-proctoring-what-it-catches-and-what-it-misses

4. Existing System Comparison

Project: Development of a Proctoring System for Online Programming Contests **Version:** prob-period-1

4.1 Purpose and method

The comparison answers one question: **does an existing system already provide what this project plans to build** — automatic judging of programming contests together with participant-side integrity control and live visibility for the organizer?

Selection. Five systems were compared: DOMjudge (the judging system this project builds on), Codeforces (the best-known contest platform), and three commercial coding-assessment platforms with proctoring features: HackerRank, CodeSignal, Codility. The set is within the required 3–5 range.

Criteria (10). The same ten criteria were applied to every system:

ID	Criterion
C1	Open source and self-hostable
C2	Automatic judging or scoring of code
C3	Contest management with a live scoreboard
C4	Detection of tab or window switching
C5	Copy and paste tracking or restriction
C6	Full-screen enforcement
C7	Integrity events visible to the organizer *during* the contest
C8	Camera, screen or ID-based proctoring
C9	Documented API for integration
C10	Code similarity or plagiarism analysis

Evidence rules. Each cell states a finding and a source tag (see 4.4). A cell that says **“not described”** means the reviewed sources do not mention the feature; it does **not** prove the feature is absent. Most sources about commercial platforms are the vendors’ own help pages or third-party guides, so they describe what the vendor says, not what was independently tested.

4.2 Comparison table

Criterion	DOMjudge	Codeforces	HackerRank	CodeSignal	Codility
C1 Open source, self-hostable	**Yes** — open-source judge, deployable by the organizer [D1, D2]	Not described (hosted platform)	Not described (hosted commercial service)	Not described (hosted commercial service)	Not described (hosted commercial service)

Criterion	DOMjudge	Codeforces	HackerRank	CodeSignal	Codility
C2 Automatic judging	**Yes** — compilation and test runs in sandboxed judgehosts, verdict categories [D1, D2]	**Yes** — solutions are judged during rounds [CF1]	**Yes** — coding tests; code behaviour analysed [H1]	**Yes** — scores calculated automatically [CS4]	**Yes** — code evolution and checks on submissions [CD1]
C3 Contest management, live scoreboard	**Yes** — jury and team interfaces, public and jury scoreboards updated in real time, freezing [D1, D2, D3]	**Yes** — contest rounds (standings not examined in detail)	Not described	Not described	Not described
C4 Tab / window switch detection	Not described	Not described	**Yes** — tab switching flagged and logged in Proctor Mode; alerts in Secure Mode [H1, H3]	Not described	**Yes** — loss of focus listed among signals (third-party source) [CD3]
C5 Copy / paste tracking or restriction	Not described	Not described	**Yes** — copy-paste tracked by default; blocked in Secure Mode [H1, H2]	Not described	**Yes** — paste detection listed among signals (third-party source) [CD3]
C6 Full-screen enforcement	Not described	Not described	**Yes** — enforced in Secure Mode and Proctor Mode [H1, H3]	Not described (entire-screen sharing is required) [CS1]	Not described
C7 Events visible to organizer during the contest	Not described	Not described	**Partly** — rules are checked automatically in real time; the organizer receives flags and reports afterwards; a live organizer view is not described [H3, H4]	**No** — the hiring company receives a score and a verification status, not recordings [CS3]	**Partly** — aggregated integrity-risk level for review; described as decision support after the test [CD1]
C8 Camera / screen / ID proctoring	Not described	Not described	**Yes** — webcam, image proctoring, desktop app mode [H1, H5]	**Yes** — camera, microphone, screen recording, photo ID [CS1, CS2]	**Yes** — webcam and screen snapshots, optional full recording, ID verification (premium) [CD1]

Criterion	DOMjudge	Codeforces	HackerRank	CodeSignal	Codility
C9 Documented API	**Yes** — documented REST API and Contest API integration [D1, D2]	Not checked	Not described	Not described	Not described
C10 Similarity / plagiarism analysis	Not described	**Yes** — post-contest detection of coinciding solutions with automatic notices [CF1, CF2]	**Yes** — AI plagiarism detection [H1]	Not described	**Yes** — similarity check and code evolution chart [CD1]

4.3 Findings

1. **DOMjudge is a strong base but has no participant-side control.** The reviewed sources describe judging, scoreboards, roles and a REST API. They do not describe tab, window, clipboard or full-screen monitoring, and they do not describe any live view of participant behaviour.
2. **Commercial platforms do track behaviour, but are closed and hiring-oriented.** HackerRank, CodeSignal and Codility describe integrity features such as tab monitoring, paste tracking, snapshots, and ID checks. Their stronger modes rely on camera, screen recording or a desktop app, which raises privacy and deployment questions for a university course.
3. **Live organizer visibility is weak or absent in the reviewed sources.** The platforms present integrity results as flags, risk levels or verification statuses for review after the test. For CodeSignal the sources say the company does not get the recordings at all.
4. **Codeforces relies on after-the-fact similarity checks.** The reviewed material shows notices about coinciding solutions sent after a round; it does not describe browser-side monitoring.
5. **Gap used to justify the project.** Within the reviewed sources, no single system combines (a) an open, self-hostable judging core, (b) browser-based integrity events without camera or screen recording, and (c) a live organizer view with an event log for reviewing disputed cases. This is the niche the proposed system targets. The statement is limited to the sources reviewed and to the date of review (October 2026).

4.4 Limits of this comparison

- “Not described” cells must be checked before the final defence. Priority checks: the **official DOMjudge documentation** (the DOMjudge facts here come from hosting-provider and university pages, not from the project’s own manual) and the **Codeforces API and rules pages**.
- HackerRank behaviour depends on the mode the employer selects; defaults differ between sources and release dates. The table lists capabilities, not defaults.
- The Codility signals in C4 and C5 come from a third-party article, not from Codility’s documentation. They should be confirmed on Codility’s help pages.
- No system was tested hands-on; the comparison is documentary.

4.5 Sources

Tag	Source
D1	DOMjudge overview (Hostinger application page): https://hostinger.com/vps/docker/domjudge
D2	DOMjudge overview (DINAO catalog): https://dinao.com/en/conteneur/domjudge
D3	DOMjudge participant guide, Florida State University: https://contest.cs.fsu.edu/static/docs/domjudge_guide.pdf
CF1	Codeforces blog post with a “significantly coincides” notice: https://codeforces.com/blog/entry/73944
CF2	Codeforces blog post with a similar notice: https://codeforces.com/blog/entry/112350
H1	HackerRank Test Integrity: https://hackerrank-knowledge-base.help.usepylon.com/articles/1079706165
H2	PracHub guide on HackerRank recording and copy-paste rules: https://prachub.com/resources/does-hackerrank-record-your-screen-proctoring-tab-switching-webcam-and-copy-paste-rules
H3	HackerRank Proctor Mode: https://hackerrank-knowledge-base.help.usepylon.com/articles/5663779659-proctor-mode
H4	HackerRank Proctor Mode (article 5663779659): https://hackerrank-knowledge-base.help.usepylon.com/articles/5663779659
H5	HackerRank Desktop App Mode: https://hackerrank-knowledge-base.help.usepylon.com/articles/5973590014
CS1	PracHub guide on CodeSignal proctoring: https://prachub.com/resources/does-codesignal-record-your-screen-proctoring-camera-and-id-rules-in-2026
CS2	CodeSignal support article on proctoring: https://support.codesignal.com/hc/en-us/articles/360039872174
CS3	Third-party guide on what the hiring company receives: https://autogpt.net/codesignal-what-candidates-should-know/
CS4	Third-party article on CodeSignal scoring and recording: https://www.interviewcoder.co/blog/does-codesignal-record-screen
CD1	Codility Proctoring & Integrity: https://www.codility.com/1p/codility-proctoring-integrity/
CD3	Third-party summary of Codility signals: https://www.vervecopilot.com/blog/can-hirevue-detect-interview-copilot

5. Technology Comparison

Project: Development of a Proctoring System for Online Programming Contests **Version:** prob-period-1

5.1 Method

Seven key technical decisions were compared, each with two or three reasonable alternatives (the requirement is at least three decisions). Every decision is judged against the requirements of section 6 (FR/NFR identifiers are given in the criteria column), not against general popularity.

Scores are the team's engineering judgement made in October 2026, before implementation: **++** strong fit, **+** adequate, **–** weak or costly for this project. They are not benchmark results. Where a claim comes from documentation, the source is cited in 5.9. Performance claims will be checked by the load test in section 9 (metric M3), not assumed here.

ID	Decision
T1	Server platform
T2	Persistent data storage
T3	Real-time channel to the teacher
T4	Client application framework
T5	In-browser code editor
T6	Integration method with DOMjudge
T7	Method of participant-side integrity control

5.2 T1 — Server platform

Needs: REST API of at least 25 endpoints with a formal contract (NFR-05), WebSocket support (FR-21), request validation, tests (NFR-04), and work in parallel with the frontend through an agreed contract.

Criterion	FastAPI (Python)	Django + DRF (Python)	Node.js (NestJS / Express)
WebSocket support	++ built in through Starlette [R1]	+ needs Channels	++ native
Contract-first API (OpenAPI)	++ OpenAPI generated from typed code	+ needs extra package	+ needs extra packages
Request/response validation	++ Pydantic models	+ serializers	+ with libraries
Async I/O for many waiting connections	++	+ (async support is partial)	++
Fit with the team's backend skills (Python)	++	++	–
Amount of ready-made features (admin, ORM)	+	++	+

Decision: FastAPI. The OpenAPI contract is the boundary between the two team members' parts, so generating it from the code removes a manual step. Django offers more ready-made features but they are not needed here; Node.js would add a second language to the team's backend stack. **Risk accepted:** FastAPI does not validate WebSocket message payloads automatically, so they are validated manually with Pydantic [R2].

5.3 T2 — Persistent data storage

Needs: relations between contests, participants, submissions and events (section 7.4), time-ordered event queries for reports (FR-23), flexible event details, transactions.

Criterion	PostgreSQL	MongoDB	MySQL
Relational integrity (foreign keys, transactions)	++	–	++
Flexible event payloads	++ JSONB column	++ documents	+ JSON column
Reporting queries (joins, aggregation by participant and time)	++	+	++
Migration tooling with the chosen stack (SQLAlchemy, Alembic)	++	–	++
Team familiarity	++	+	+

Decision: PostgreSQL for persistent data, **plus Redis** for short-lived data (timer cache, publish/subscribe between backend workers, rate limiting). Redis is not an alternative to PostgreSQL here: it holds nothing that must survive a restart. MySQL is a close second; PostgreSQL was preferred for its JSONB support for event details.

Is Redis strictly necessary for the timer? For correctness, no. The authoritative values (`starts_at`, `duration_seconds`, `extra_seconds`, and therefore `ends_at`) are stored in PostgreSQL, and every submission is checked against them there, so the system stays correct without Redis. Redis is added for three reasons that follow from the requirements:

1. **Read load of timer synchronization.** Each client re-synchronizes its clock periodically (section 7.6). With 50 participants, and up to 200 connections in the stress test (section 9), the contest end time would be read from PostgreSQL on every call. A cache serves these reads from memory; PostgreSQL is touched only when the teacher extends or finishes the contest.
2. **Fan-out between workers.** WebSocket connections are spread across several backend workers, and an event or timer change accepted by one worker must reach teacher connections held by another. Redis publish/subscribe does this. PostgreSQL `LISTEN/NOTIFY` is a possible alternative; it was not chosen because it ties message delivery to database connections, which is the resource we want to keep free under load.
3. **Rate limiting** of event intake per participant.

If Redis is unavailable, timer reads fall back to PostgreSQL and the contest continues without live push. Whether the cache is worth its cost will be checked in the load test (section 9, metric M3): the same test

is run with and without the cache, and if the difference is negligible, Redis is limited to fan-out and rate limiting.

5.4 T3 — Real-time channel from the server to the teacher

Needs: teacher sees a violation within 2 s (NFR-02), the contest timer remains synchronized (NFR-01), connection loss is survivable.

Criterion	WebSocket	Server-Sent Events (SSE)	HTTP polling
Delay of event delivery	++ immediate push	++ immediate push	– bounded by polling interval; a 2 s target needs polling more often than every 2 s
Two-way messages (acknowledgements, heartbeats, timer sync)	++	– one-way only	+ via separate requests
Server load for 50+ clients	+ one connection each	+ one connection each	– many repeated requests
Reconnection handling	+ must be written	++ built into the browser API	++ trivial
Support in the chosen backend	++ [R1]	+	++

Decision: WebSocket for the teacher's live monitoring and for the participant's session channel (heartbeat and timer synchronization). Participant events themselves are sent by ordinary HTTPS requests in batches, so they are not lost when the socket drops. SSE would be sufficient for one-way push to the teacher; WebSocket was chosen because the same technology also covers the two-way participant heartbeat. Polling is kept as the **fallback** if the connection cannot be established.

5.5 T4 — Client application framework

Needs: at least 12 screens for two roles (NFR-06), real-time updates, localization into three languages (FR-24), typed code shared with the API contract.

Criterion	React + TypeScript (Vite)	Vue 3 + TypeScript	Angular
Fit with the frontend developer's plan and skills	++	+	+
Ready-made editor integrations	++	+	+
Internationalization libraries (i18next)	++	++	++
Learning and setup cost in the available time	+	+	– larger framework

Decision: React with TypeScript, built with Vite, as stated in the supervisor's endorsement. The three options are comparable technically; the deciding factors are the developer's skills and the available editor integrations.

5.6 T5 — In-browser code editor

Needs: syntax highlighting for C++, Python, Java (FR-07), control over the clipboard and context menu (FR-17), draft autosave (FR-08).

Criterion	Monaco Editor	CodeMirror 6	Plain textarea
Syntax highlighting for the three languages	++	++	– none
Interception of copy, paste, cut and context menu	+ needs overriding editor commands	++ lighter and easier to extend	++ simple DOM events
Familiarity for participants (resembles a desktop IDE)	++	+	–
Bundle size	– large	++ small	++

Decision: Monaco Editor. It gives participants an interface close to common IDEs. The cost is a larger bundle and extra work to override its clipboard commands; this is a risk for FR-17 and is covered by the violation-capture test (metric M5). CodeMirror 6 is the planned fallback if Monaco's clipboard handling cannot be controlled reliably.

5.7 T6 — Integration with DOMjudge

Needs: contests, problems, submissions, verdicts and scoreboard (FR-04, FR-09, FR-10, FR-11) without modifying DOMjudge.

Criterion	DOMjudge REST API	Direct access to DOMjudge's database	Command-line submit script
Supported, documented interface	++ [D1]	– internal schema may change	+
Does not modify DOMjudge	++	– risk of corrupting its data	++
Full set of operations (read verdicts, scoreboard)	++	++	– submit only
Safe across DOMjudge upgrades	+	–	+

Decision: DOMjudge REST API, accessed only through one backend module (the DOMjudge adapter, section 7). This keeps the integration replaceable and testable.

5.8 T7 — Participant-side integrity control

Needs: detect tab or window change, leaving full screen, copy and paste (FR-15 – FR-17) without camera or screen recording (NFR-10), in the participant's ordinary browser.

Criterion	Browser APIs (Page Visibility, Fullscreen, Clipboard events)	Browser extension	Locked-down desktop application
Installation by participants	++ none	– must install per browser	– must install per operating system
Strength of control	– can detect, cannot guarantee prevention	+ wider access to browser	++ operating-system level; comparable products use it for their strictest mode [H5]
Privacy impact	++ low	+	– high
Development effort for a two-person project	++	–	–
Works without administrator rights	++	+	–

Decision: browser APIs, with the model **"prevention plus recording"**: block what can be blocked, record what cannot, and validate and score events on the server. The Page Visibility API fires when the user switches tabs, minimizes the browser or switches app [R3]. This approach cannot stop a determined user (for example one using a second device); this is a stated limit of the project (section 9.8), and the desktop-application option is listed as a possible future extension.

5.9 Summary of decisions

ID	Chosen	Main reason	Verified by
T1	FastAPI	OpenAPI contract, WebSocket, Python skills	Contract tests, M8
T2	PostgreSQL + Redis	Relational integrity, JSONB, fast ephemeral data	Reports, integration tests
T3	WebSocket (HTTPS for events, polling as fallback)	Push within 2 s, two-way channel	M2, M3
T4	React + TypeScript + Vite	Developer skills, ecosystem	M9
T5	Monaco Editor	IDE-like experience	M5
T6	DOMjudge REST API	Supported interface, replaceable	Integration tests
T7	Browser APIs	No installation, low privacy impact	M5, M10

Decisions will be recorded as architecture decision records (ADR) in `docs/adr/` when implementation starts, and revisited if measurements contradict the assumptions above.

5.10 Sources

Tag	Source
R1	FastAPI documentation, WebSockets: https://fastapi.tiangolo.com/reference/websockets/

Tag	Source
R2	Note on manual payload validation for FastAPI WebSockets (community documentation copy): https://huggingface.co/spaces/Nomearod/agentbench/blob/main/data/tech_docs/fastapi_websockets.md
R3	MDN, `visibilitychange` event: https://developer.mozilla.org/docs/Web/Events/visibilitychange
D1	DOMjudge overview (REST API): https://dinao.com/en/conteneur/domjudge
H5	HackerRank Desktop App Mode: https://hackerrank-knowledge-base.help.usepylon.com/articles/5973590014

Claims in the comparison tables that have no tag (for example the effort of learning Angular, or Django’s partial async support) are the team’s judgement and should be treated as such.

6. Requirements Specification

Project: Development of a Proctoring System for Online Programming Contests **Version:** prob-period-1

6.1 Scope and conventions

The system has two user roles, **student** (participant) and **teacher** (organizer), and uses **DOMjudge** as the judging backend (section 7). These are the initial requirements for the probationary period; they will be refined after interviews (section 2) and supervisor feedback (section 11).

- **Priority (MoSCoW): Must** — needed for the minimum viable system and for the pilot; **Should** — important, planned for MVP-2; **Could** — only if time remains.
- **Source of each requirement:** SE = supervisor's endorsement of the project (stated functions and target values); TC = team's decision derived from the problem analysis. Requirements will additionally be tagged with interview evidence once section 2 is complete; until then they are **assumptions that have not yet been verified with users**.
- **Owner:** B = backend (Nurkhan), F = frontend (Bakytzhan), B+F = both.
- Each requirement has an acceptance criterion that can be checked by a test, a measurement, or an inspection. Measurement procedures are in section 9 (metrics M1–M10).

6.2 Functional requirements

Accounts and roles

ID	Requirement	Priority	Source	Owner	Acceptance criterion
FR-01	A user logs in with e-mail and password and receives a JWT; the role is student or teacher	Must	SE	B+F	Valid credentials give access; invalid ones return an error; a student's token is rejected by every teacher-only endpoint (automated role test)
FR-02	A teacher creates and edits a contest: title, start time, duration, problems	Must	SE	B+F	A created contest appears in the contest list with the entered values and in DOMjudge
FR-03	A teacher adds participants to a contest (by e-mail or list import) and removes them	Must	TC	B+F	Added participants see the contest; removed ones no longer do

Contest and DOMjudge integration

ID	Requirement	Priority	Source	Owner	Acceptance criterion
FR-04	The system synchronizes contests and problems with DOMjudge	Must	SE	B	After creation, the contest and its problems exist in DOMjudge and the identifiers are stored; sync is repeatable without duplicates
FR-05	A student sees available contests and joins a running one	Must	TC	B+F	A student sees only their contests and can open a running one; cannot open a contest they are not part of
FR-06	A student reads the problem statement	Must	SE	B+F	The statement of every problem of the contest is displayed with its limits
FR-09	A submission is sent to DOMjudge for judging	Must	SE	B	A submitted solution reaches DOMjudge and receives a verdict that matches DOMjudge’s own record
FR-10	A student sees the history of own submissions with verdicts	Must	SE	B+F	Every submission is listed with time, language and final verdict; a student cannot see others’ submissions
FR-11	A scoreboard is shown to students and the teacher	Should	SE	B+F	Scoreboard matches DOMjudge’s scoreboard for the same contest

ID	Requirement	Priority	Source	Owner	Acceptance criterion
FR-25	The system keeps working when DOMjudge is temporarily unavailable	Should	TC	B	With DOMjudge stopped, a submission is stored as pending and retried; the user sees a status message; no submission is lost

Editor and timer

ID	Requirement	Priority	Source	Owner	Acceptance criterion
FR-07	The code editor highlights syntax for C++, Python and Java, with language choice and code templates	Must	SE	F	Each of the three languages is highlighted and selectable; template inserted on request
FR-08	The draft of the solution is saved automatically and restored after reload	Should	SE	F+B	After closing and reopening the page, the last saved draft is restored
FR-12	The server is the only source of contest time; it starts the contest and closes submissions automatically at the end	Must	SE	B	A submission sent after the end time is rejected with a clear error, independent of the client clock
FR-13	A teacher can extend or finish a contest	Should	TC	B+F	Extension is reflected for all participants' timers within the NFR-01 limit
FR-14	The client shows a timer synchronized with server time and blocks submission at the end	Must	SE	F	Timer shows remaining time; the submit button is disabled at zero; the timer survives reload and short network loss

Participant-side integrity control

ID	Requirement	Priority	Source	Owner	Acceptance criterion
FR-15	The client detects tab and window switching during a contest	Must	SE	F	Switching tab, window or application produces an event in the server log
FR-16	The client requires full-screen mode and detects leaving it	Must	SE	F	Leaving full screen produces an event and a request to return
FR-17	The editor blocks copy, paste, cut and the context menu and reports attempts	Must	SE	F	Each attempt is blocked and produces an event
FR-18	The client sends events to the server; the server validates, timestamps and stores them	Must	SE	B	Events from a participant of a running contest are stored with server time; events with invalid contest, role or time are rejected
FR-19	The participant receives a visible warning when an event is recorded	Should	SE	F	A warning appears within 1 s of the event, unless the teacher disabled warnings
FR-20	The server calculates a risk score for each participant from the event log	Should	SE	B	The score is deterministic for a given log; a participant with no events has score 0; score definition is documented

Teacher monitoring and reports

ID	Requirement	Priority	Source	Owner	Acceptance criterion
FR-21	The teacher sees violations and risk scores of participants in real time	Must	SE	B+F	A scripted event appears on the teacher's dashboard within the NFR-02 limit
FR-22	The teacher reviews incidents: opens the event log, marks an incident reviewed or dismissed, adds a note	Should	TC	B+F	Status and note are saved and shown in later reports
FR-23	The teacher exports a contest report (results, events, incidents)	Should	SE	B	A CSV or JSON file is produced containing every participant, verdicts and events of the contest

Localization

ID	Requirement	Priority	Source	Owner	Acceptance criterion
FR-24	The interface is available in Russian, Kazakh and English	Could	SE	F	Every screen can be switched between the three languages without untranslated text

6.3 Non-functional requirements

ID	Requirement	Priority	Source	Owner	Acceptance criterion (metric in section 9)
NFR-01	Timer discrepancy between client and server	Must	SE	B+F	≤ 1 s, including after reload and short network loss (M1)
NFR-02	Delivery time of a proctoring event to the teacher	Must	SE	B+F	≤ 2 s (M2)

ID	Requirement	Priority	Source	Owner	Acceptance criterion (metric in section 9)
NFR-03	Number of simultaneous participants	Must	SE	B	≥ 50 participants starting together without errors above the agreed threshold; stress test up to 200 concurrent connections, results reported (M3)
NFR-04	Automated test coverage of key modules	Should	SE	B+F	≥ 70% (M4)
NFR-05	API size and documentation	Must	SE	B	≥ 25 endpoints, all described in OpenAPI, contract tests pass (M8)
NFR-06	Size of the client application	Should	SE	F	≥ 12 screens for the two roles (M9)
NFR-07	Usability	Should	SE	F	SUS score ≥ 70 (M6); participant can start and submit without instructions (M7)
NFR-08	Share of scripted violations recorded	Must	SE	B+F	≥ 90% of a fixed violation script (M5)
NFR-09	Security	Must	TC	B	Passwords are stored hashed; communication over HTTPS/WSS; role checks on every endpoint; events from another participant or a finished contest are rejected; replayed or forged events are detected in the bypass-resistance test

ID	Requirement	Priority	Source	Owner	Acceptance criterion (metric in section 9)
NFR-10	Privacy	Must	TC	B+F	No camera, microphone or screen recording; only the event types of FR-15 – FR-17 are collected; participants are told what is recorded before the contest starts
NFR-11	Supported browsers	Must	TC	F	Works in current versions of Chrome, Firefox and Edge; behaviour per browser is reported
NFR-12	Deployment	Should	SE	B	The whole system (backend, database, cache, DOMjudge test stand) starts from one Docker Compose command following the README
NFR-13	Recoverability during a contest	Must	SE	B+F	After page reload or a connection break, the participant returns to the same contest with correct remaining time and the saved draft
NFR-14	Maintainability	Should	TC	B+F	Continuous integration runs the tests on every pull request; no secrets in the repository
NFR-15	No unfair penalty for honest participants	Must	SE	B+F	A scripted honest session produces no violation (M10)

6.4 Traceability

Requirement group	Components (section 7)	Verification (section 9)
FR-01 – FR-03	Auth and roles, contest service, student UI, teacher dashboard	Role and permission tests
FR-04, FR-09 – FR-11, FR-25	DOMjudge adapter, submission service	Integration tests with the DOMjudge stand
FR-07, FR-08	Code editor, API client	End-to-end tests, M9
FR-12 – FR-14, NFR-01, NFR-13	Timer service, timer widget, Redis	M1, end-to-end tests
FR-15 – FR-17, FR-19	Proctoring module	M5, M10
FR-18, FR-20 – FR-23, NFR-02	Event intake, risk scoring, monitoring hub, report service	M2, M5
NFR-03	Whole backend	M3
NFR-05, NFR-04	REST API layer, test suite	M8, M4
NFR-07	Prototype and client	M6, M7

6.5 Out of scope for this stage

Camera, microphone or screen recording; automatic punishment of participants (the teacher makes the final decision); a mobile application; implementation of a new judging engine (judging is delegated to DOMjudge); preventing the use of a second device.

6.6 Open points

- Exact definition of the risk score (FR-20) and the thresholds for “low”, “medium”, “high” will be fixed after the first measurements; the dashboard prototype uses illustrative values.
- Priorities of FR-11, FR-19, FR-22 and FR-24 should be confirmed with the supervisor and with interview results.
- Requirements will be marked **verified** or **assumption** once the evidence of relevance (section 2) is complete.

7. Architecture and Diagrams

Project: Development of a Proctoring System for Online Programming Contests **Version:** prob-period-1 **Status:** draft architecture for the probationary period; the component diagram is a draft, as the requirements specify.

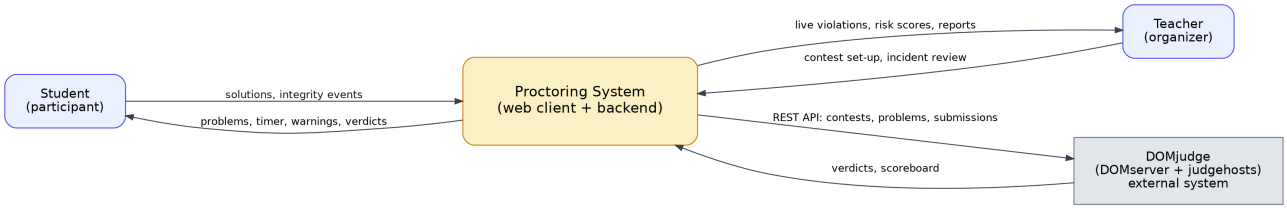
Editable sources of all diagrams are in the repository folder `diagrams/` (Graphviz `.dot` and Mermaid `.mmd` files), with PNG exports next to them.

7.1 Architectural principles

1. **Judging is delegated to DOMjudge.** The system does not implement a judge; it adds a participant client, integrity control and an organizer dashboard around DOMjudge (T6 in section 5).
2. **Two contours with a fixed contract.** The web client (frontend) and the backend communicate only through the REST API described in OpenAPI, plus two WebSocket channels. This is the boundary between the two team members' areas.
3. **The server is the source of truth** for time and for recorded events. The client reports; it is never trusted to decide.
4. **Prevention plus recording.** The browser blocks what it can; everything else is recorded, validated and scored on the server (T7).
5. **One module talks to DOMjudge.** Replacing or mocking the judge affects only the DOMjudge adapter.

7.2 Diagrams

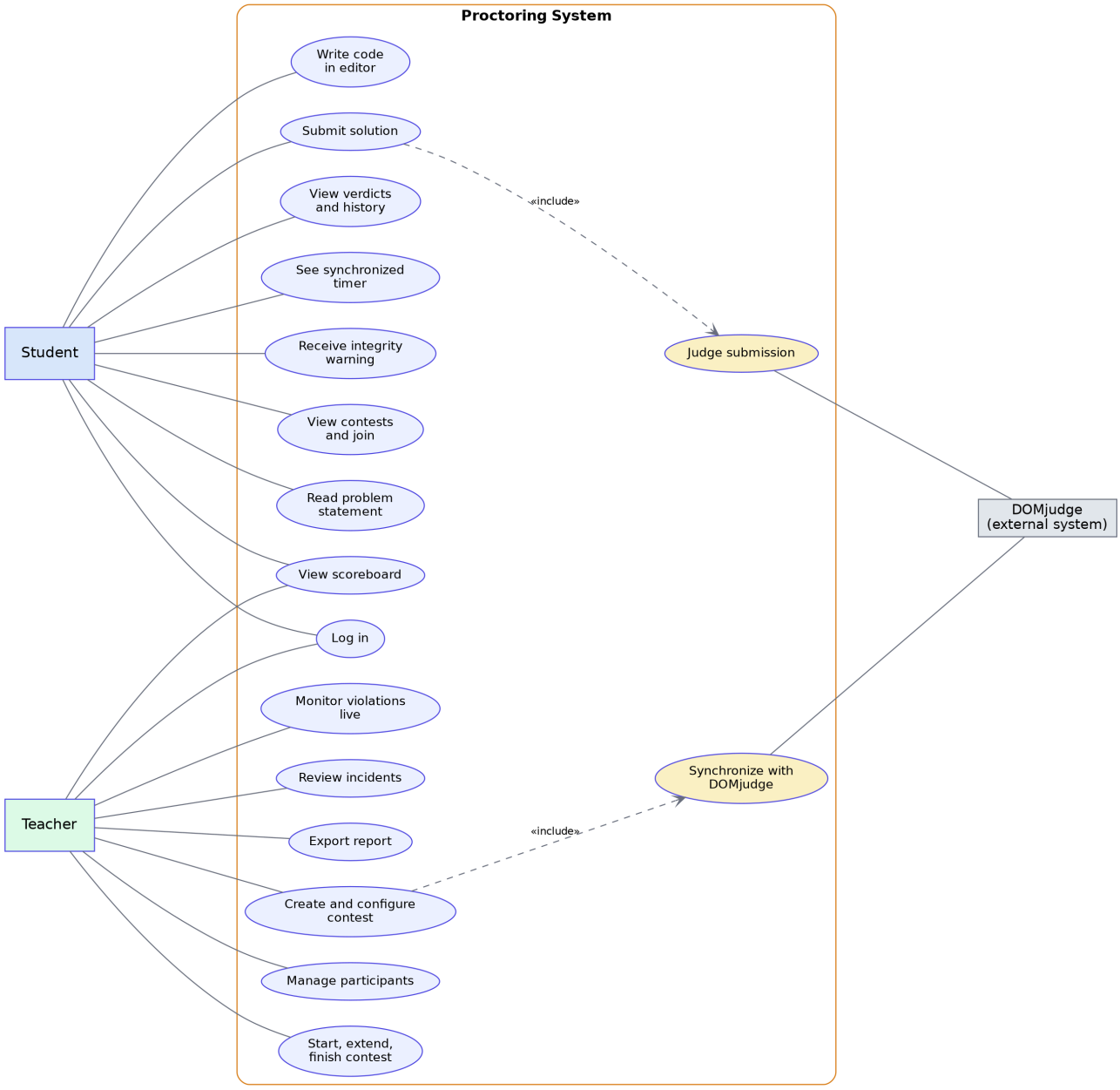
7.2.1 System context



System context diagram

The system has two human actors, the student and the teacher, and one external system, DOMjudge. DOMjudge receives submissions and contest definitions, and returns verdicts and the scoreboard.

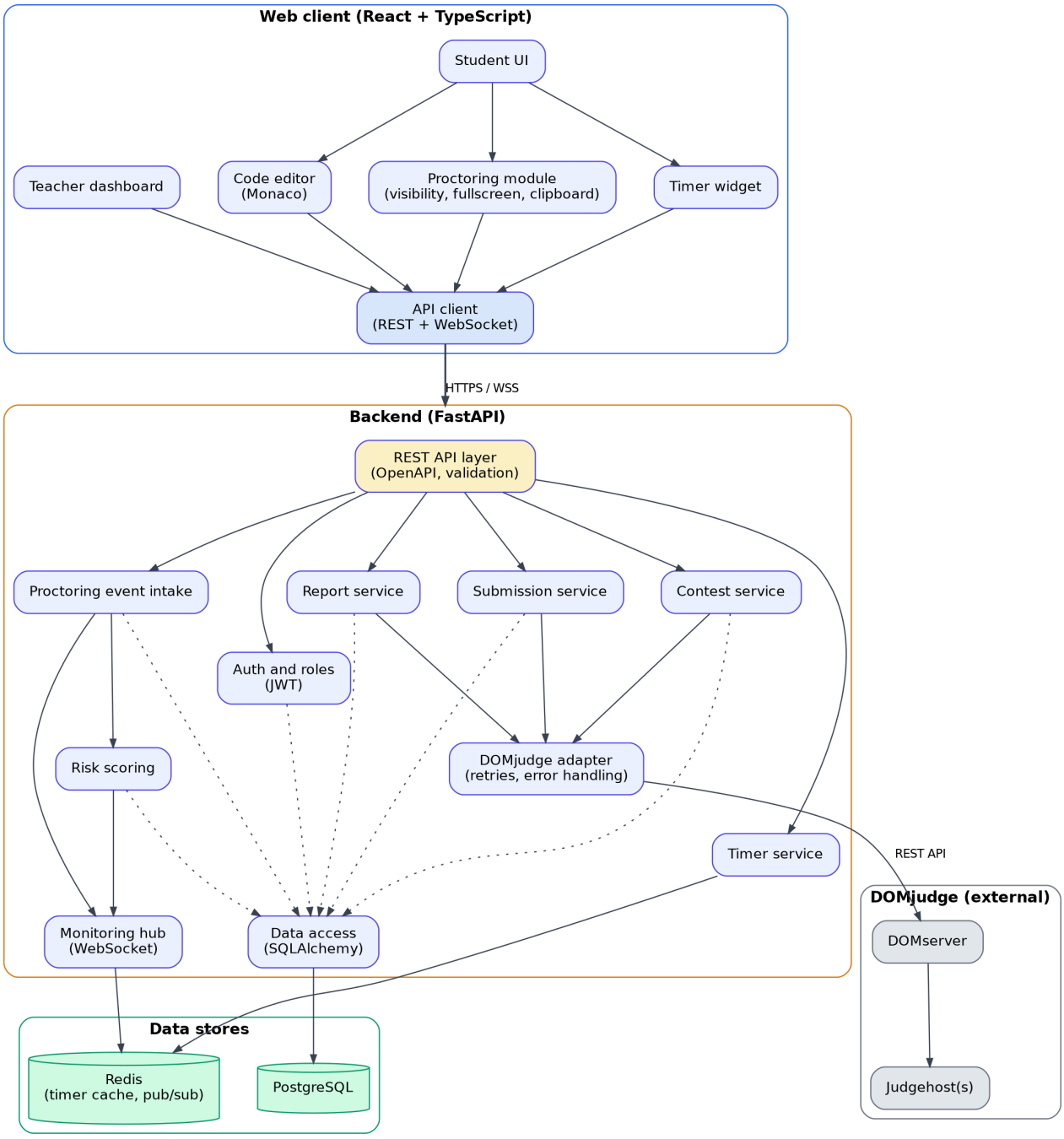
7.2.2 Use case diagram



Use case diagram

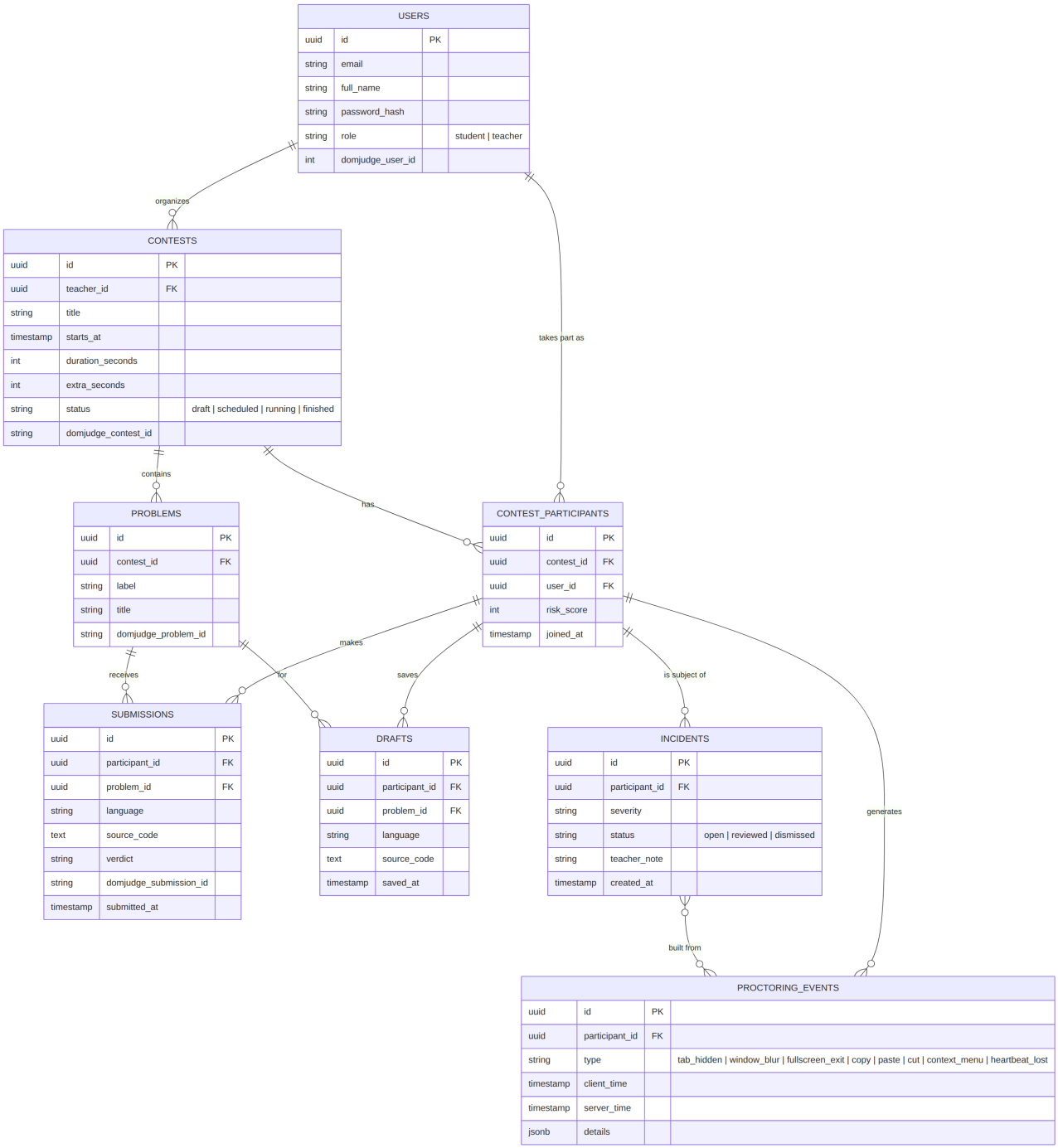
The student reads problems, writes and submits code, sees the timer and receives integrity warnings. The teacher creates and controls contests, monitors violations live, reviews incidents and exports reports. Two use cases involve DOMjudge: judging a submission and synchronizing a contest.

7.2.3 Component diagram (draft)



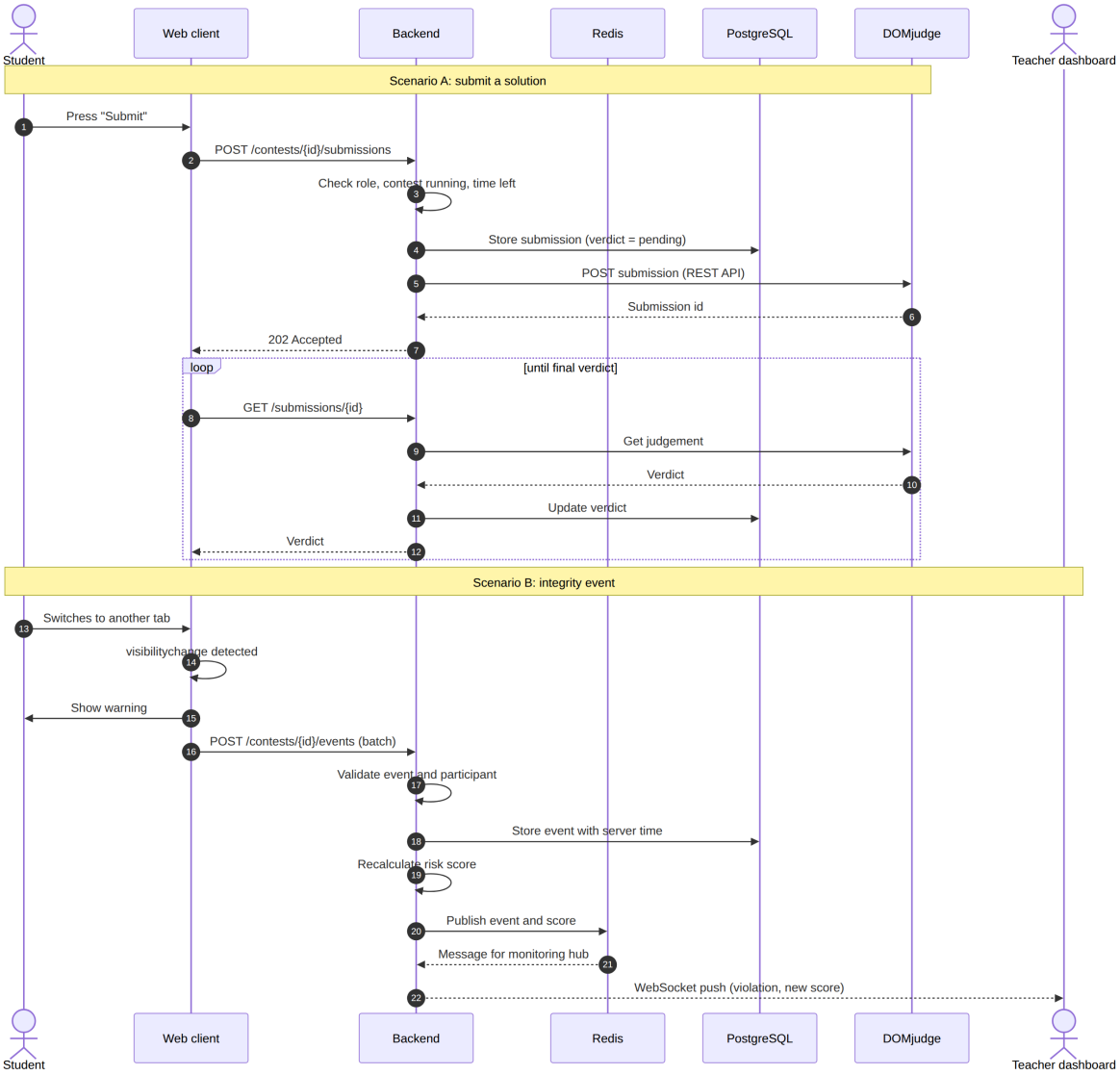
Component diagram

7.2.4 Data model



Entity-relationship diagram

7.2.5 Key scenarios



Sequence diagram: submission and integrity event

7.3 Components and responsibilities

Component	Responsibility	Requirements
Student UI	Contest list, problem page, submission history, warnings	FR-05, FR-06, FR-10, FR-19
Code editor (Monaco)	Syntax highlighting, language choice, templates, clipboard blocking, draft autosave	FR-07, FR-08, FR-17
Proctoring module	Detects tab/window change, full-screen exit, copy/paste/cut/context menu; batches events and sends them	FR-15, FR-16, FR-17
Timer widget	Shows remaining time using server-synchronized offset; disables submission at zero	FR-14, NFR-01

Component	Responsibility	Requirements
Teacher dashboard	Contest set-up, participant list, live feed, incident review, report	FR-02, FR-03, FR-13, FR-21 – FR-23
API client	REST and WebSocket access, reconnection, token handling	NFR-13
REST API layer	Routing, validation, OpenAPI generation, error format	NFR-05
Auth and roles	Login, JWT, role checks on every endpoint	FR-01, NFR-09
Contest service	Contest and participant lifecycle, status changes	FR-02 – FR-05, FR-13
Submission service	Accept submissions, check contest state and time, store, track verdicts	FR-09, FR-10
DOMjudge adapter	All calls to DOMjudge with retries, timeouts, error mapping	FR-04, FR-09, FR-11, FR-25
Timer service	Authoritative start/end time, remaining-time endpoint, automatic closing	FR-12, FR-13
Proctoring event intake	Validates and stores events with server time; detects sequence gaps and lost heartbeat	FR-18, NFR-09
Risk scoring	Computes participant risk score from the event log	FR-20
Monitoring hub	Pushes events and scores to teachers over WebSocket via Redis publish/subscribe	FR-21, NFR-02
Report service	Aggregates results, events and incidents; exports CSV/JSON	FR-23
PostgreSQL	Persistent data	section 7.4
Redis	Timer cache, publish/subscribe between workers, rate limiting	NFR-01, NFR-02, NFR-03

7.4 Data model

The entities and their relations are shown in 7.2.4. Design notes:

- A **contest participant** is a separate entity linking a user to a contest, so that submissions, drafts, events and incidents belong to a participation, not directly to an account.
- **Proctoring events** store both `client_time` and `server_time`; reports use `server_time`. Event-specific data (for example how long the page was hidden) goes into a JSON field.
- **Incidents** are created by the risk logic or by the teacher from a group of events, and carry the teacher's decision (`open`, `reviewed`, `dismissed`) and note.
- Identifiers of DOMjudge objects (`domjudge_contest_id`, `domjudge_problem_id`, `domjudge_submission_id`, `domjudge_user_id`) are stored so that synchronization is repeatable without duplicates.

- Personal data is limited to name, e-mail and the event log (NFR-10).

7.5 API outline (REST, base path `/api/v1`)

The contract will be written as an OpenAPI specification in `docs/api/openapi.yaml` and kept as the single source of truth for both team members. The outline below has **35 REST endpoints and 2 WebSocket channels**, above the target of 25 (NFR-05).

#	Method and path	Role	Purpose	Requirement
1	POST `/auth/login`	any	Log in, receive tokens	FR-01
2	POST `/auth/refresh`	any	Refresh access token	FR-01
3	POST `/auth/logout`	any	End session	FR-01
4	GET `/users/me`	any	Current user and role	FR-01
5	POST `/users`	teacher	Create a user	FR-03
6	GET `/users`	teacher	List users	FR-03
7	GET `/contests`	any	Contests visible to the user	FR-05
8	POST `/contests`	teacher	Create contest	FR-02
9	GET `/contests/{id}`	any	Contest details	FR-05
10	PATCH `/contests/{id}`	teacher	Edit contest	FR-02
11	POST `/contests/{id}/sync`	teacher	Synchronize with DOMjudge	FR-04
12	POST `/contests/{id}/start`	teacher	Start contest	FR-12
13	POST `/contests/{id}/extend`	teacher	Extend duration	FR-13
14	POST `/contests/{id}/finish`	teacher	Finish contest	FR-13
15	GET `/contests/{id}/participants`	teacher	List participants	FR-03
16	POST `/contests/{id}/participants`	teacher	Add participants	FR-03

#	Method and path	Role	Purpose	Requirement
17	DELETE `/contests/{id}/participants/{user_id}`	teacher	Remove participant	FR-03
18	POST `/contests/{id}/join`	student	Enter a running contest	FR-05
19	GET `/contests/{id}/problems`	any	List problems	FR-06
20	GET `/contests/{id}/problems/{problem_id}`	any	Problem statement	FR-06
21	GET `/contests/{id}/problems/{problem_id}/draft`	student	Read saved draft	FR-08
22	PUT `/contests/{id}/problems/{problem_id}/draft`	student	Save draft	FR-08
23	POST `/contests/{id}/submissions`	student	Submit solution	FR-09
24	GET `/contests/{id}/submissions`	any	Own submissions (student) or all (teacher)	FR-10
25	GET `/submissions/{id}`	any	Submission with verdict	FR-10
26	GET `/contests/{id}/scoreboard`	any	Scoreboard	FR-11
27	GET `/contests/{id}/time`	any	Server time, end time, remaining	FR-12, FR-14
28	POST `/contests/{id}/events`	student	Send batch of proctoring events	FR-18
29	GET `/contests/{id}/events`	teacher	Event log with filters; `after` parameter returns events newer than a given server time (catch-up after reconnection)	FR-21, NFR-13

#	Method and path	Role	Purpose	Requirement
30	GET `/contests/{id}/participants/{user_id}/risk`	teacher	Risk score and its components	FR-20
31	GET `/contests/{id}/incidents`	teacher	List incidents	FR-22
32	PATCH `/incidents/{id}`	teacher	Review, dismiss, add note	FR-22
33	GET `/contests/{id}/report`	teacher	Contest report	FR-23
34	GET `/contests/{id}/report/export`	teacher	Export CSV or JSON	FR-23
35	GET `/health`	public	Service health check	NFR-12
WS-1	`/ws/contests/{id}/monitor`	teacher	Live events and scores	FR-21
WS-2	`/ws/contests/{id}/session`	student	Heartbeat and timer synchronization	FR-14, FR-18

Examples of requests and responses are in section 8.

7.6 Design of the critical mechanisms

Server timer (FR-12, FR-14, NFR-01)

The server stores `ends_at = starts_at + duration + extra`. The client calls `GET /contests/{id}/time`, measures the round-trip time, and computes its clock offset from the server time and half of the round trip. It repeats the measurement periodically, after every reconnection and when the tab becomes visible again. The timer shown to the participant is `ends_at − (local time + offset)`. Submission is rejected by the server after `ends_at` regardless of what the client shows. The synchronization period is a tuning parameter to be set by measurement (metric M1).

WebSocket reconnection (FR-14, FR-18, FR-21, NFR-13)

Both WebSocket channels (WS-1 teacher monitoring, WS-2 student session) can drop because of network loss, a sleeping laptop or a server restart. The client treats this as a normal event and handles it as follows:

1. **Detection.** The server sends a ping every 5 s. The client closes and reconnects if no message or pong arrives within 15 s, or immediately when the browser reports a `close` or `error` event.
2. **Reconnection.** Exponential backoff with random jitter (about 1 s, 2 s, 4 s, up to a cap of 15 s), so that dozens of clients do not reconnect at the same instant after a server restart.
3. **Re-authentication.** An expired access token is refreshed before reconnecting.

4. **Catch-up.** The teacher dashboard requests missed events with `GET /contests/{id}/events?after=<server time of the last received event>`. The student client re-reads `GET /contests/{id}/time` to resynchronize the timer and sends the integrity events buffered while offline; unsent events are kept in memory and in browser storage until the server acknowledges them.
5. **Fallback.** If reconnection fails for 30 s, the client switches to polling `GET /contests/{id}/time` and `GET /contests/{id}/events?after=` every 2 s and keeps trying to restore the WebSocket in the background.
6. **Visibility.** The interface shows a connection indicator. When the session heartbeat stops, the teacher sees the participant as offline and the server records a `heartbeat_lost` event (see below).

The intervals and the cap are initial values, to be tuned in the reconnection test (section 9).

Event handling (FR-15 – FR-18, FR-20, NFR-09)

The client batches events and sends them over HTTPS; each event carries a per-participant sequence number and the client timestamp. The server checks that the contest is running and that the participant belongs to it, assigns server time, and stores the event. Because the client code runs on the participant's machine, the server does not rely on it alone:

- **sequence gaps** and duplicate numbers are flagged;
- if the heartbeat on the session channel stops for longer than a set interval, the server records a `heartbeat_lost` event, which also covers the case where client checks were disabled;
- request rate is limited per participant.

Risk score (FR-20) — initial proposal

The score is computed from the event log by weighting event types (for example, a paste attempt counts more than a context-menu click), merging events caused by the same action, and capping the total at 100. Levels (low, medium, high) are set by thresholds. **Weights and thresholds are placeholders** to be calibrated on the first measurements and on the pilot; the score supports the teacher's decision and never penalizes automatically.

7.7 Deployment view

Part	Technology	Note
Web client	Static build served over HTTPS	React, Vite
Backend	FastAPI application, several workers	Stateless; shared state in Redis and PostgreSQL
Database	PostgreSQL	Alembic migrations
Cache and messaging	Redis	Timer cache, publish/subscribe, rate limits
Judge	DOMserver and one or more judgehosts	Test stand deployed by the team
Packaging	Docker and Docker Compose	One command brings up the whole stack (NFR-12)

Part	Technology	Note
Automation	GitHub Actions or GitLab CI	Tests on every pull request (NFR-14)

7.8 Security and privacy

Passwords are stored as hashes; tokens are short-lived with refresh; HTTPS and WSS are used throughout; every endpoint checks the role and ownership (a student cannot read another participant's data); the repository holds no secrets (placeholders in configuration examples). Only the event types of FR-15 – FR-17 and the heartbeat are collected; there is no camera, microphone or screen capture, and participants are informed before the contest starts (NFR-10). The retention period of event logs is to be agreed with the supervisor.

7.9 Consistency check

Check	Result
Every Must requirement of section 6 is covered by at least one component in 7.3	Yes (traceability in 6.4)
Every endpoint in 7.5 refers to a requirement	Yes
Every entity in the ER diagram is used by at least one endpoint	Users: 1–6; contests: 7–14; participants: 15–18; problems: 19–20; drafts: 21–22; submissions: 23–25; events: 28–29, WS; incidents: 31–32
Both scenarios of the sequence diagram (submit, integrity event) map to endpoints and components	Submit: endpoints 23, 25 → submission service → DOMjudge adapter. Event: endpoint 28 → intake → risk scoring → monitoring hub → WS-1
Technology choices in section 5 appear in the component and deployment views	Yes (FastAPI, PostgreSQL, Redis, WebSocket, React, Monaco, DOMjudge REST API, browser APIs)

7.10 Known limits and next steps

- The risk score weights, the retention period and the exact event batching interval are not fixed yet.
- The OpenAPI specification still has to be written from the outline in 7.5; this is the first task of the next stage and is needed before the two parts diverge.
- The component diagram is a draft and will be refined when the DOMjudge test stand reveals the real shape of its API responses.

8. UX / Prototype

Project: Development of a Proctoring System for Online Programming Contests **Version:** prob-period-1

8.1 Status and links

This section shows how users and the external system will interact with the solution: low-fidelity mockups of six key screens for the two roles, and examples of the REST API calls behind them.

Item	Value
Interactive prototype (Figma)	Figma prototype
Mockup sources	`prototype/src/*.html` (editable), PNG exports in `prototype/`
Status	Draft wireframes. **They have not yet been tested with users.** A test with 5 respondents is planned (section 9, metrics M6 and M7)

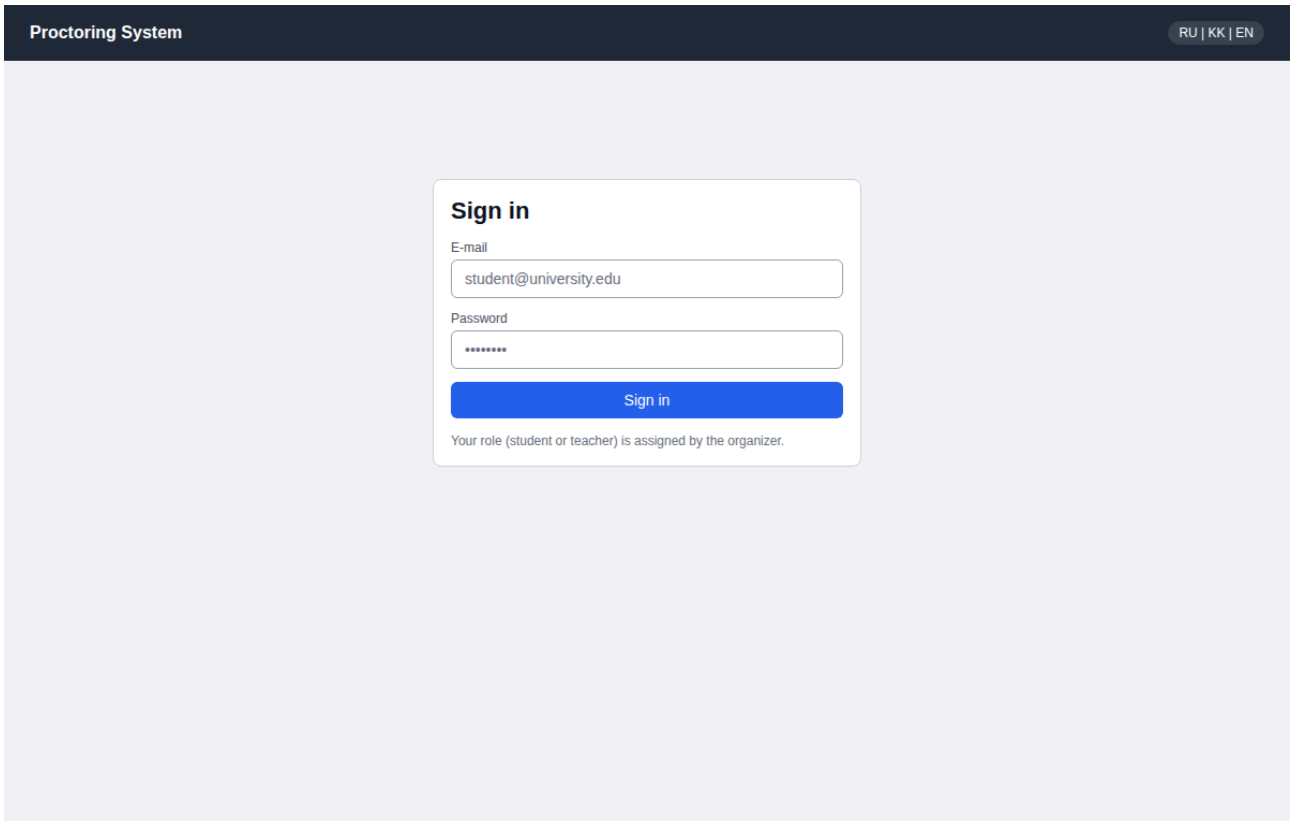
All names, times and risk values in the mockups are illustrative.

8.2 Key scenarios

#	Scenario	Role	Screens	Related requirements
UX-1	Log in	student, teacher	Screen 1	FR-01
UX-2	Find a contest and enter it, after reading what is recorded	student	Screen 2	FR-05, NFR-10
UX-3	Solve a problem: read the statement, write code, submit, see the verdict, watch the timer	student	Screen 3	FR-06 – FR-10, FR-14
UX-4	Receive a warning after leaving the contest tab	student	Screen 4	FR-15, FR-19
UX-5	Create a contest and choose proctoring settings	teacher	Screen 5	FR-02, FR-03, FR-04
UX-6	Monitor participants live and review an incident	teacher	Screen 6	FR-21, FR-22

8.3 Mockups

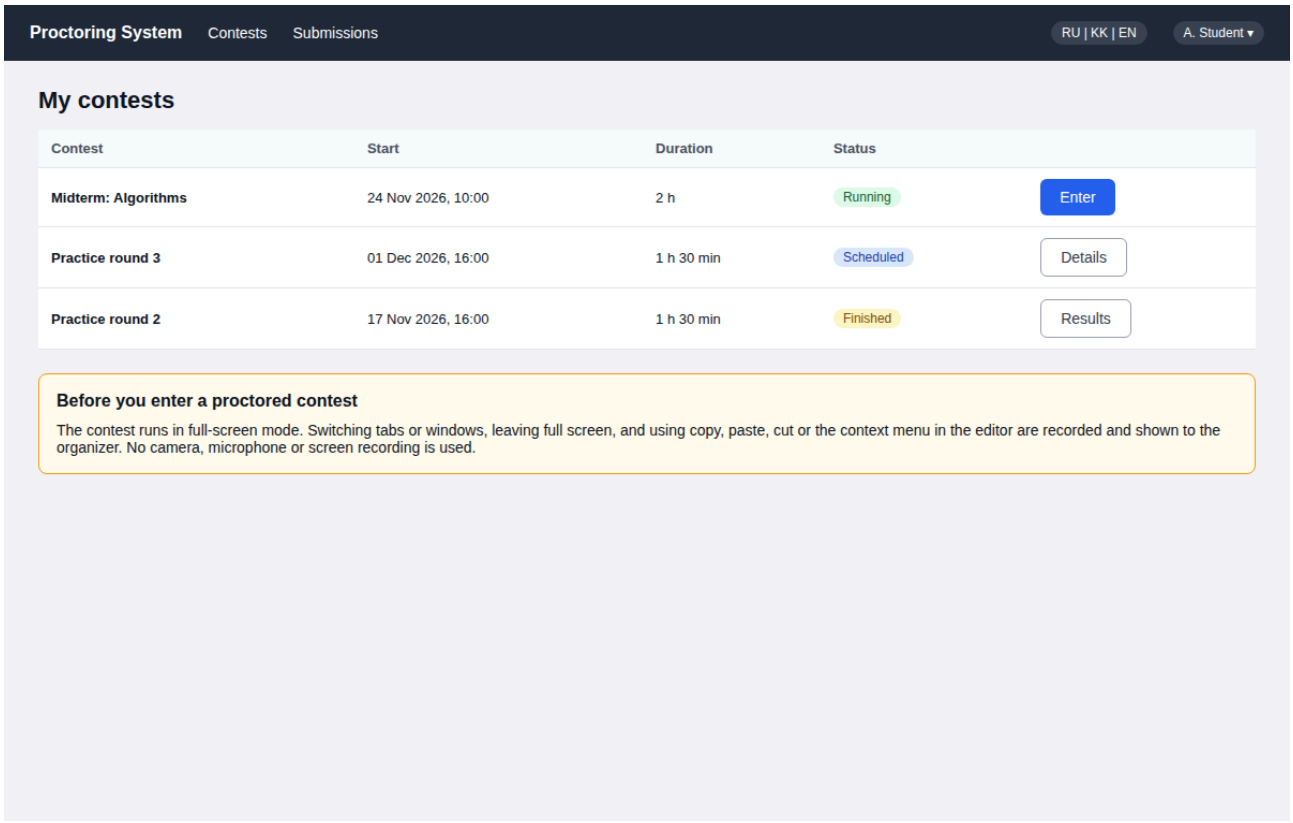
Screen 1 — Log in



Login screen

One form for both roles; the role is assigned by the organizer. Language switch (RU, KK, EN) is in the top bar (FR-24).

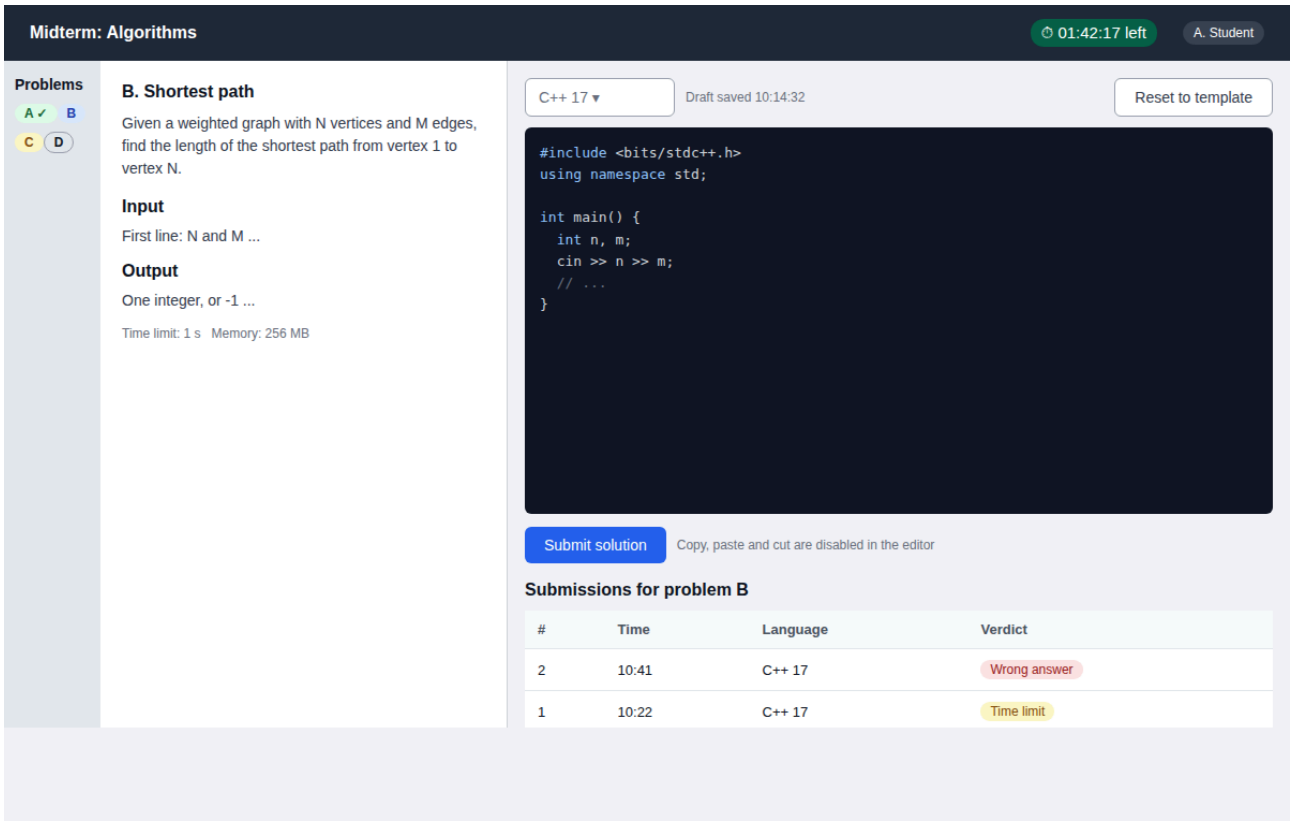
Screen 2 — Student: contest list



Contest list

Status tags show whether a contest is running, scheduled or finished. Before entering, the student sees a plain-language notice of what is recorded and what is not (no camera, microphone or screen recording), which supports NFR-10 and the pilot’s consent requirement.

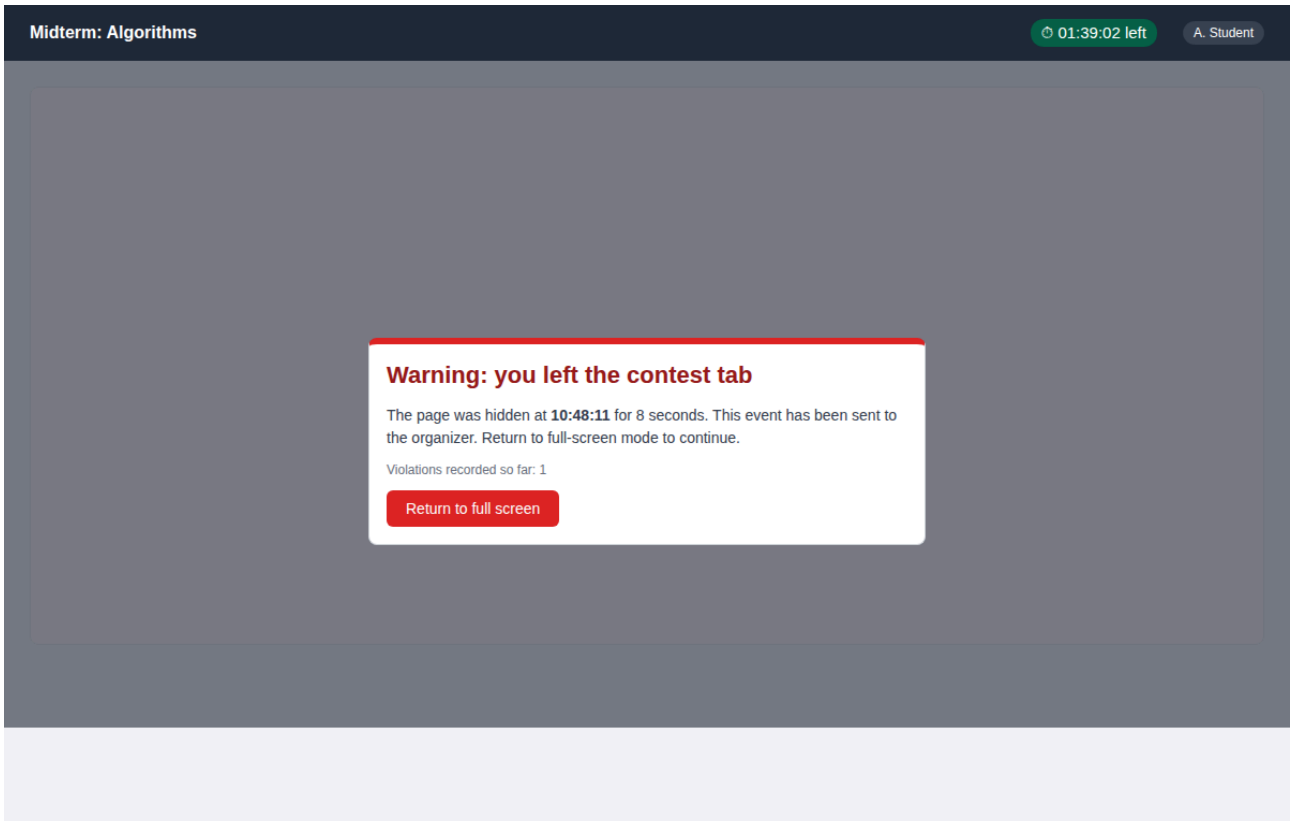
Screen 3 — Student: problem, editor and timer



Problem page

Three areas: problem list with solved marks, statement, and editor with language choice, draft status and submission history. The remaining time is always visible in the top bar. The note under the editor tells the student that copy, paste and cut are disabled, so the block is expected rather than surprising.

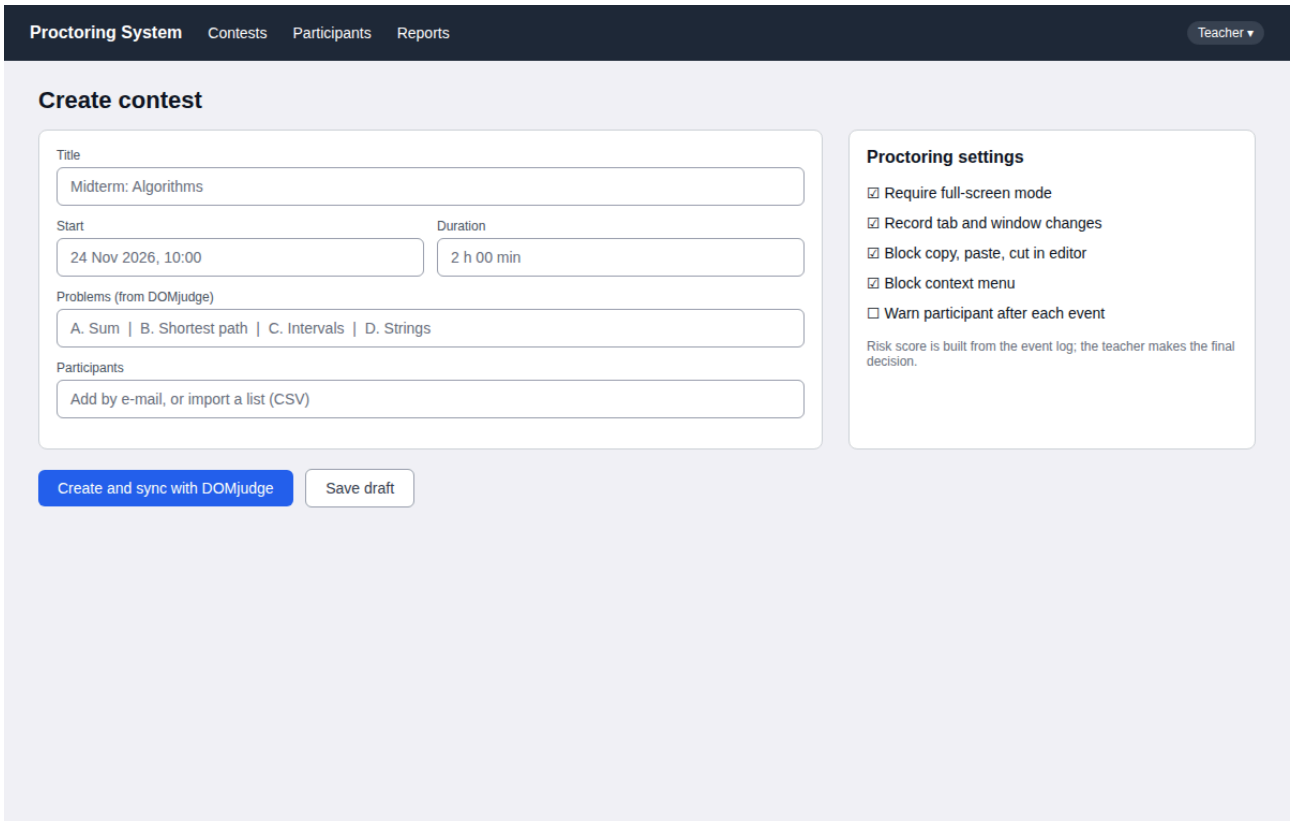
Screen 4 — Student: integrity warning



Warning dialog

After a tab or window change the student sees what happened, when, and that it was reported, and can return to full-screen mode with one action. The text avoids accusation: it states the event, not a conclusion.

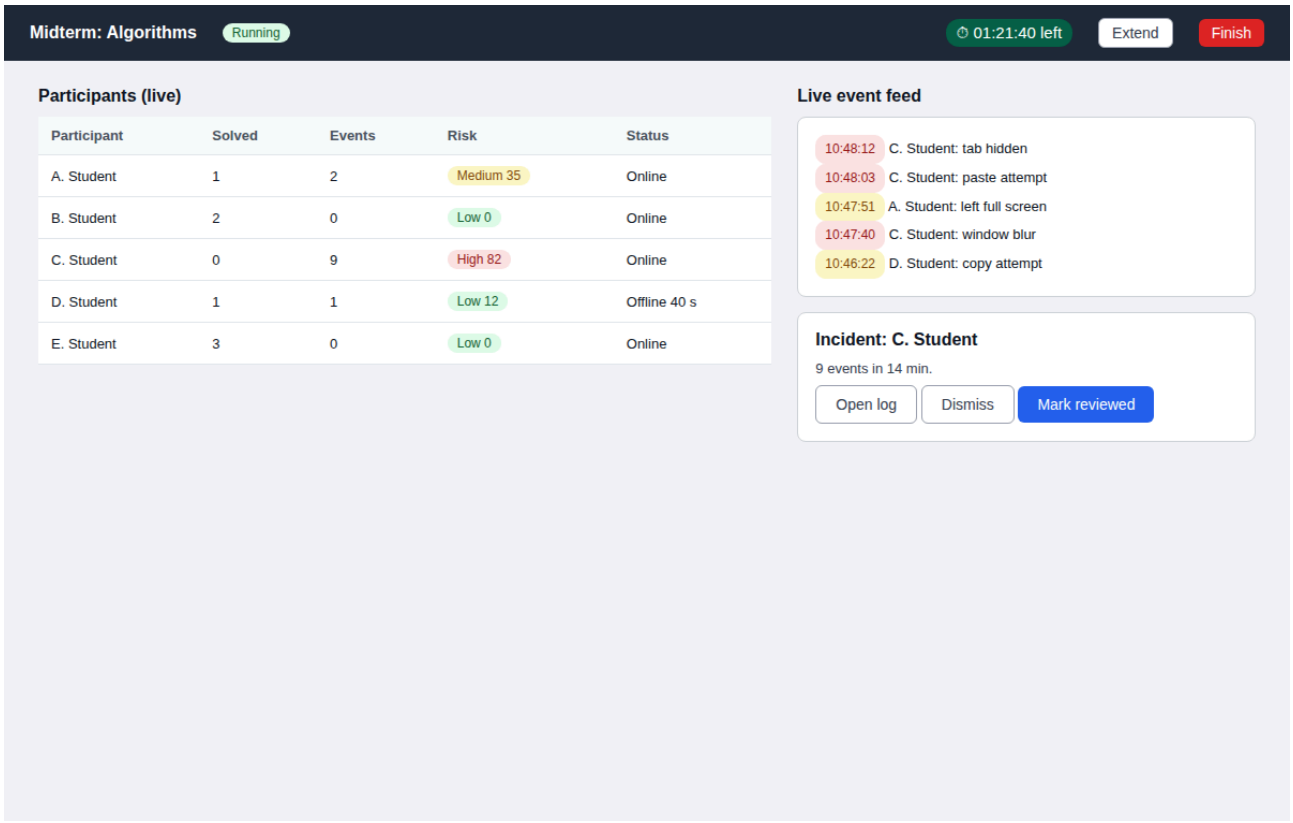
Screen 5 — Teacher: create a contest



Create contest

The teacher enters the contest data, selects problems from DOMjudge, adds participants, and chooses which integrity events are recorded and whether warnings are shown. A single action creates the contest and synchronizes it with DOMjudge.

Screen 6 — Teacher: live monitoring



Live monitoring dashboard

A participant table (solved problems, number of events, risk level, online status), a live event feed, and an incident panel where the teacher opens the log, dismisses or marks the incident reviewed. The risk level is shown with its number so the teacher can see how close a participant is to a threshold; the final decision stays with the teacher.

8.4 API examples

These examples show how the client and an external tool interact with the backend. Field names are a proposal and will be fixed in the OpenAPI specification.

Log in — `POST /api/v1/auth/login`

Request:

```
{ "email": "student@university.edu", "password": "********" }
```

Response `200`:

```
{
  "access_token": "<jwt>",
  "refresh_token": "<token>",
  "expires_in": 900,
  "user": { "id": "7c1f…", "full_name": "A. Student", "role": "student" }
}
```

Server time — `GET /api/v1/contests/{id}/time`

Response `200`:

```
        {
          "server_time": "2026-11-24T10:14:32.418Z",
          "starts_at": "2026-11-24T10:00:00Z",
          "ends_at": "2026-11-24T12:00:00Z",
          "remaining_seconds": 6327,
          "status": "running"
        }
```

Submit a solution — `POST /api/v1/contests/{id}/submissions`

Request:

```
        { "problem_id": "b3a9…", "language": "cpp17", "source_code": "#include
<bits/stdc++.h> …" }
```

Response `202`:

```
        { "submission_id": "e41d…", "verdict": "pending", "submitted_at": "2026-11-
24T10:41:07Z" }
```

Error after the contest ended, `409`:

```
        { "error": "contest_finished", "message": "Submissions are closed for this
contest." }
```

Verdict — `GET /api/v1/submissions/{id}`

Response `200`:

```
        { "submission_id": "e41d…", "problem": "B", "language": "cpp17", "verdict":
"wrong-answer", "judged_at": "2026-11-24T10:41:19Z" }
```

Send integrity events — `POST /api/v1/contests/{id}/events`

Request:

```
        {
          "events": [
            { "seq": 41, "type": "tab_hidden",  "client_time": "2026-11-
24T10:48:03.100Z", "details": { "hidden_ms": 8200 } },
            { "seq": 42, "type": "paste",       "client_time": "2026-11-
24T10:48:12.900Z", "details": { "blocked": true } }
          ]
        }
```

Response `202`:

```
{ "accepted": 2, "rejected": 0 }
```

Live message to the teacher — WebSocket `/ws/contests/{id}/monitor`

```
{
  "type": "event",
  "participant": { "user_id": "9a02…", "full_name": "C. Student" },
  "event": { "type": "tab_hidden", "server_time": "2026-11-24T10:48:03.140Z" },
  "risk": { "score": 82, "level": "high" }
}
```

Contest report — `GET /api/v1/contests/{id}/report/export?format=csv`

The file has one row per participant: name, problems solved, penalty, number of events by type, risk score, incident status and teacher note.

8.5 How the prototype will be evaluated

- **Usability test with 5 respondents** on the Figma prototype: task “find the contest, read problem B, submit, understand the warning”, observation notes, and the SUS questionnaire (targets M6 ≥ 70, M7 unassisted use).
- **Comparison with baseline B0:** the standard DOMjudge team interface on the same task (section 9.5).
- Findings will be recorded as tasks in YouTrack and changes will go through reviewed pull requests.

8.6 Known limits

- The mockups are low-fidelity; visual design, accessibility and responsive behaviour are not designed yet.
- Only the six key scenarios are shown; the full application is planned to have at least 12 screens (NFR-06), including the scoreboard, submission history, participant management, incident details and the report view.
- Texts of warnings and the consent notice must be reviewed with the supervisor.

9. Verification and Evaluation Plan

Project: Development of a Proctoring System for Online Programming Contests **Team:** Nurkhan Kuanyshov (backend), Bakytzhan Dzhangazy (frontend, UX/UI), group SE-2431 **Supervisor:** Abzal Ermekbayuly Kyzyrkanov **Version:** prob-period-1

9.1 Purpose and status

This plan defines how the team will demonstrate that the project aim is achieved: a web system that lets a teacher run an online programming contest on top of DOMjudge and receive reliable data about participants' integrity-related behaviour.

At the probationary period 1 stage **no measurements have been performed yet**. This section fixes in advance what will be measured, under which conditions, against which baseline, and what counts as success. The target values come from the supervisor's endorsement of the project; the measurement procedures and acceptance thresholds not stated there are the team's proposals and are marked as such.

9.2 Outcome metrics

ID	Metric	Target	How it is measured	Owner
M1	Timer discrepancy between client and server	≤ 1 s	Automated browser test samples the timer shown to the participant and the server time at regular intervals during a full contest run, including after page reload and after a short network interruption. Reported value: maximum absolute difference.	Nurkhan (server timer), Bakytzhan (client display)
M2	Delivery time of a proctoring event to the teacher	≤ 2 s	Time from the browser event (client timestamp) to its appearance in the teacher's real-time view. Reported: maximum and 95th percentile.	Nurkhan, Bakytzhan

ID	Metric	Target	How it is measured	Owner
M3	Concurrent participants supported	≥ 50 required; stress test up to 200 concurrent connections reported	Load test: 50 simulated participants start at the same moment, submit solutions and send events. Proposed acceptance thresholds: error rate below 1% and M2 still met. The load is then raised to 100, 150 and 200 concurrent connections (participants plus teacher dashboards) and the error rate, event delivery time (M2) and timer error (M1) are reported at each step. No pass threshold is set above 50; the aim is to find the limit and the first bottleneck. The run is repeated with and without the Redis cache (section 5.3).	Nurkhan
M4	Automated test coverage of key modules	≥ 70%	Coverage reports from pytest (backend) and Vitest (frontend). Key modules: authentication and roles, timer, event intake, DOMjudge integration.	Both
M5	Share of scripted violations recorded	≥ 90%	A fixed script of violations (tab switch, window switch, leaving full screen, copy, paste, cut, context menu) is performed repeatedly. Capture rate = events stored on the server / actions performed.	Both

ID	Metric	Target	How it is measured	Owner
M6	System Usability Scale (SUS) score	≥ 70	Standard 10-item SUS questionnaire completed by pilot participants and by the 5 respondents testing the Figma prototype.	Bakytzhan
M7	Unassisted use	Participant starts and submits a solution without instructions	Observed task: log in, open a problem, write and submit a solution. Reported: share of participants who complete it without help.	Bakytzhan
M8	API size and documentation	≥ 25 endpoints in OpenAPI	Count of endpoints in the generated OpenAPI specification; contract tests pass against it.	Nurkhan
M9	Client application size	≥ 12 screens for two roles	Inventory of implemented screens against the prototype.	Bakytzhan
M10	Disruption of honest participants	No violation flagged during a scripted honest session (proposed)	A script of normal work (typing, running, submitting, scrolling the statement) is executed; any warning or incident is counted as a false positive.	Both

9.3 Testing plan

Level	What is checked	Tools	Planned period
Unit tests	Business logic: timer calculation, risk score, event validation, role checks	pytest, Vitest	From MVP-1 (December 2026)
Integration tests	Backend with PostgreSQL, Redis and a running DOMjudge test stand: contests, problems, submissions, verdicts, scoreboard	pytest, Docker Compose	December 2026 – February 2027
Contract tests	REST API against the OpenAPI specification shared by both contours	OpenAPI-based validation	From the API contract (November 2026)

Level	What is checked	Tools	Planned period
Role and permission tests	A student cannot reach teacher endpoints; a teacher sees only their own contests	pytest	MVP-1
End-to-end tests	Key scenarios in a real browser: login, solve, submit, timer expiry, violation appears in teacher panel	Playwright	MVP-2 (January – February 2027)
Reconnection test	Drop and restore the connection (network off, server restart): timer resynchronization (M1), no lost events (M5), catch-up on the teacher dashboard, fallback to polling	Playwright with network throttling, manual server restart	February – March 2027
Proctoring behaviour tests	Detection and server-side recording of scripted violations (M5), false-positive check (M10)	Playwright, manual script	February – March 2027
Load test	Simultaneous start and submission by 50 participants (M3), then stress steps up to 200 concurrent connections, event latency under load (M2), with and without the Redis cache	k6	March 2027
Bypass-resistance check	Attempts to forge events, replay requests, disable client checks; findings documented as limitations	Manual, scripted requests	March 2027
Usability testing	Prototype with 5 respondents; SUS and observation in the pilot (M6, M7)	Figma, SUS questionnaire	November 2026 (prototype), March 2027 (pilot)
Pilot contest	Full contest with a group of students, including volunteers who perform the scripted violations	The deployed system	March 2027

9.4 Measurement conditions

These conditions will be recorded with every reported result so that measurements can be repeated:

- **Environment:** DOMjudge, backend, PostgreSQL and Redis deployed with Docker Compose; machine specification (CPU, memory, operating system) and software versions written down for each run.

- **Browsers:** current versions of Chrome, Firefox and Edge. Browser behaviour differs for the Page Visibility, Fullscreen and Clipboard APIs, so M5 is reported per browser.
- **Network:** a local network run, and a degraded run with added latency and packet loss (proposed values: 100 ms latency, 1% loss). Timer recovery (M1) is checked with a deliberate connection break.
- **Repetition:** each measurement is repeated at least three times; the median and the range are reported.
- **Version:** every result states the Git tag or commit it was obtained on.

9.5 Baseline for comparison

Baseline B0 — DOMjudge used on its own, with its standard team interface and no proctoring layer.

Comparison	Baseline B0	Proposed system
Integrity events visible to the organizer (tab switch, leaving full screen, copy and paste)	None expected, because the participant-side behaviour is outside DOMjudge's scope. **This claim must be checked against the official DOMjudge documentation before it is stated in the final report.**	Measured by M5
Usability for a participant	SUS of the standard DOMjudge team interface, measured on the same task	SUS of the new client (M6)
Real-time visibility for the teacher	Scoreboard and submissions only	Violations and risk score in real time (M2)

The baseline is a comparison of functions and usability, not of judging quality: both solutions use DOMjudge for judging, so verdict correctness is expected to be identical and is only checked for consistency.

9.6 Pilot contest design

- **Participants:** a group of students; the size and date are to be agreed with the supervisor. Minimum useful composition: honest participants and a few volunteers who follow the violation script.
- **Consent and data:** participants are told which events are recorded (tab and window changes, full-screen exit, copy and paste attempts). No camera, microphone or screen recording is used. Personal data is kept to what the contest needs, and results are reported in aggregate.
- **Data collected:** event logs, timer samples, submission verdicts, SUS questionnaires, observation notes.
- **Evidence kept in the repository:** raw measurement files, scripts that produced the numbers, and the final summary table.

9.7 How the aim will be demonstrated

Part of the project aim	Evidence
The system works with DOMjudge	Integration tests and the pilot contest with real verdicts and a scoreboard (M8, integration tests)
The timer is reliable	M1 results, including recovery after reload and network loss
Violations are detected and shown to the teacher	M5 and M2 results
The system is usable without training	M6 and M7 results compared with baseline B0
The system works at class scale	M3 results
The result is of engineering quality	M4 coverage, contract tests, bypass-resistance findings documented as limitations

9.8 Known limits of the evaluation

- A browser cannot guarantee that every action is blocked, so the project evaluates *prevention plus recording*, not complete prevention. Use of a second device is outside the measured scope and is documented as a limitation.
- The pilot group is small, so SUS and capture-rate results are indicative, not statistically conclusive.
- Targets M1–M5 are the supervisor-endorsed values; if measurements show they cannot be met, the report will state the achieved values and the cause rather than adjust the targets afterwards.

10. Project Management and Risks

Project: Development of a Proctoring System for Online Programming Contests **Team:** Nurkhan Kuanyshov, Bakytzhan Dzhangazy (group SE-2431) **Supervisor:** Abzal Ermekbayuly Kyzyrkanov, assistant professor **Period:** October 2026 – May 2027

10.1 Git repository

Item	Value
Repository	https://github.com/Nurkhoop/proctoring-system-diploma
Tag for this submission	`prob-period-1` — https://github.com/Nurkhoop/proctoring-system-diploma/releases/tag/prob-period-1
Visibility and access	Public repository; both team members commit from their own accounts
Main branch	`main` is the reviewed baseline

Repository structure

Folder / file	Content
`README.md`	Title, problem, users, team and roles, scope, status, technologies, links, setup instructions when code is runnable
`docs/`	Proposal, evidence of relevance, literature review, system and technology comparisons, requirements, architectural decisions, verification plan, management and risks, supervisor feedback
`diagrams/`	Editable diagram sources and readable exports
`src/`	Application code by component (backend, frontend)
`tests/`	Automated tests and supporting materials
`config/`	Dependency definitions and configuration examples with placeholder values
`.gitignore`	Excludes secrets, keys and personal data

Working rules

- Every meaningful work item has a YouTrack task. Branch name contains the task ID, for example `DIP-23-database-schema`; commit messages start with it, for example `DIP-23: Add initial database schema and ER diagram`.
- Changes go through a pull request that explains the change, references the task and describes how the work was checked. For this team project the other team member reviews before merging.
- Each student commits from their own account with identifiable author details; no shared accounts.
- Passwords, API keys and personal data are never committed; configuration examples use placeholders.

- Submitted tags are never moved or replaced. Later submissions get their own tags (`pre-defense-1`, `pre-defense-2`, `final-pre-defense`).

10.2 YouTrack project

Item	Value
Project	Proctoring Diploma, key `DIP`
URL	https://proctoring-diploma.youtrack.cloud/projects/DIP
Template	Kanban, with Assignee, Priority, Stage and Due date fields
Members	Both students and the supervisor

Each task states the responsible student, the expected outcome, acceptance criteria and a deadline. A task is moved to the final stage only when its acceptance criteria are met, and the link to the pull request or commit is added to the task.

10.3 Allocation of responsibilities

Areas of responsibility (from the supervisor's endorsement)

Student	Role	Area
Nurkhan Kuanyshov	Backend: system core	REST API and data model, roles and authentication, DOMjudge integration, server timer, intake and storage of proctoring events, risk score, real-time monitoring, reports, testing and deployment
Bakytzhan Dzhangazy	Frontend, UX/UI	Interfaces for both roles, code editor, copy/paste prohibition, full-screen and tab control, timer, teacher dashboard, localization, testing

The boundary between the two parts is fixed by the REST API contract (OpenAPI).

Task allocation for the first submission (all due by 24 October 2026)

ID	Task	Responsible
DIP-1	Project proposal and scope	Nurkhan
DIP-2	Evidence of relevance: interviews	Bakytzhan
DIP-3	Literature review	Bakytzhan
DIP-4	Existing systems comparison	Nurkhan
DIP-5	Technology comparison	Nurkhan
DIP-6	Requirements specification	Bakytzhan
DIP-7	Architecture and diagrams	Nurkhan
DIP-8	UX prototype	Bakytzhan

ID	Task	Responsible
DIP-9	Verification and evaluation plan	Bakytzhan
DIP-10	Project management and risks	Bakytzhan
DIP-11	Supervisor feedback	Bakytzhan
DIP-12	Repository setup and README	Nurkhan
DIP-13	Consolidated report PDF	Nurkhan
DIP-14	Tag and form submission	Nurkhan

Joint work covers requirements, overall architecture and the API contract, integration testing, the pilot contest and the defence. Where a section covers both parts (requirements, verification), each student writes the part belonging to their area.

10.4 Milestones

Period	Work	Checkpoint result
October 2026	Topic refinement, DOMjudge and analogue analysis, requirements	Requirements and user stories; first submission (`prob-period-1`)
November 2026	Architecture, data model, REST API contract, interface prototype, DOMjudge test stand	Architecture documents, data schema, OpenAPI, prototype, working stand
December 2026	MVP-1: roles, authentication, contests and problems, submission to DOMjudge, code editor	MVP-1 demonstration, first-stage report
January – February 2027	MVP-2: server timer, client proctoring, events and monitoring, teacher dashboard, scoreboard	MVP-2 demonstration, draft of main chapters
March 2027	Testing, load tests, bypass checks, pilot contest	Test reports, pilot participants' feedback
April 2027	Final measurements, results chapter, thesis text, originality check, pre-defence	Full text, pre-defence protocol
May 2027	Revisions after comments, presentation, defence	Admission to defence, working system

Supervision: a weekly team meeting of 45–60 minutes, individual consultations on request, a monthly demonstration of working functionality, and code review as pull requests arrive.

10.5 Dependencies

Dependency	Why it matters	How it is handled
DOMjudge and its REST API	Judging, verdicts and scoreboard come from it	Test stand deployed early; integration isolated behind one backend module; API version recorded

Dependency	Why it matters	How it is handled
REST API contract between the two parts	Backend and frontend work in parallel	OpenAPI agreed in November 2026 and changed only through reviewed pull requests
Browser APIs (Page Visibility, Fullscreen, Clipboard)	Client proctoring relies on them	Behaviour tested per browser; limits documented
Interviewees and pilot participants	Evidence of relevance and pilot results	Contacts arranged early; supervisor helps with the pilot group
Supervisor review	Feedback is needed for section 11 and for decisions	Draft sent in advance with a stated deadline
Hosting for the pilot	Pilot needs a reachable deployment	Docker-based deployment; hosting agreed with the university or the supervisor
YouTrack Cloud and Figma	Task evidence and prototype	Free plans; share links checked from an external account

10.6 Risks

Probability and impact are the team’s own estimates (L = low, M = medium, H = high).

ID	Risk	P	I	Mitigation	Owner
R1	A browser cannot reliably block all actions, so prevention is incomplete	H	M	Model of prevention plus recording; server-side validation; limits stated in the report	Both
R2	DOMjudge integration is harder than expected or its API differs from the documentation	M	H	Early test stand; integration module with retries and error handling; fallback to a reduced set of integration features	Nurkhan
R3	The timer drifts or breaks after reload or network loss	M	H	Single server time source, client synchronization, recovery tests (M1)	Nurkhan

ID	Risk	P	I	Mitigation	Owner
R4	False accusations: legitimate behaviour is flagged as a violation	M	H	Risk score based on an event log, not a single event; human review by the teacher; false-positive check (M10)	Both
R5	Start-of-contest load exceeds capacity	M	M	Load test with 50 users (M3); caching with Redis; load tuning	Nurkhan
R6	Events are forged or replayed by a participant	M	H	Server-side validation and checks; bypass-resistance tests	Nurkhan
R7	The two parts drift apart because the API contract changes	M	M	OpenAPI as the single source; contract tests; reviewed changes only	Both
R8	Schedule slip in a two-person team	M	H	Weekly meetings; if behind by more than a week, an individual plan with checkpoints every 3–4 days; critical functions first	Both
R9	Not enough interviewees or pilot participants	M	M	Contact people early; use the supervisor’s network; report honestly how many discussions took place	Bakytzhan
R10	Personal data of participants is handled improperly	L	H	Record only events needed for the contest; no camera or screen capture; aggregated reporting; participants informed	Both

ID	Risk	P	I	Mitigation	Owner
R11	Committee or supervisor cannot open a link or repository	M	H	Access granted before submission; links checked in a private browser window or from an external account	Nurkhan
R12	Unverified source claims in the literature review or comparison	M	M	Each student checks sources assigned to them; claims without evidence are marked as unverified	Both

10.7 Next steps

1. Complete the evidence of relevance (interviews) and record verified findings separately from assumptions.
2. Receive the supervisor’s feedback on the first-submission materials and record the decisions in section 11.
3. Fix the OpenAPI contract and start the DOMjudge test stand (November 2026).
4. Deliver the prototype for review by 5 respondents.
5. Start MVP-1 (December 2026) with every change linked to a YouTrack task and reviewed by the other team member.

11. Supervisor Feedback

Supervisor: Abzal Ermekbayuly Kyzyrkanov, assistant professor **Version:** prob-period-1

This section records the supervisor’s feedback on the key deliverables and what the team decided to do about each comment. Entries are added **only from real, documented feedback** (message, e-mail or meeting notes).

11.1 Materials sent for review

#	Deliverable	Sent on	Channel
1	Project proposal and scope (section 1)	19.10.2026	Telegram
2	Literature review and comparisons (sections 3–5)	20.10.2026	Telegram
3	Requirements and architecture (sections 6–7)	23.10.2026	Telegram
4	Verification plan and project management (sections 9–10)	24.10.2026	Telegram

11.2 Feedback and team decisions

At the time of this submission no written feedback from the supervisor on these materials has been recorded in the repository. Feedback received after submission will be added below, one row per comment, with the team’s decision and a link to the commit or pull request that contains the change.

#	Date	Deliverable	Supervisor’s comment	Team decision	Change made (task, commit or PR)	Status

11.3 Summary of how feedback changed the work

Not applicable yet: no feedback has been recorded. This subsection will be written when the first comments are received.

Appendix A. Links

All repository links point to the tag `prob-period-1`, so they show exactly the version submitted.

A.1 Project resources

Resource	Link
Git repository	https://github.com/Nurkhoop/proctoring-system-diploma
Version submitted (tag `prob-period-1`)	https://github.com/Nurkhoop/proctoring-system-diploma/tree/prob-period-1
Task tracker (YouTrack, project DIP)	https://proctoring-diploma.youtrack.cloud/projects/DIP
Interactive prototype (Figma)	Proctoring System UX Prototype
UI mockups (PNG and HTML sources)	https://github.com/Nurkhoop/proctoring-system-diploma/tree/prob-period-1/prototype

A.2 Documents

№	Document	Link
1	Project proposal	docs/01-project-proposal.md
2	Evidence of relevance	docs/02-evidence-of-relevance.md
3	Literature review	docs/03-literature-review.md
4	Existing system comparison	docs/04-systems-comparison.md
5	Technology comparison	docs/05-technology-comparison.md
6	Requirements specification	docs/06-requirements-specification.md
7	Architecture and diagrams	docs/07-architecture-and-diagrams.md
8	UX / prototype	docs/08-ux-prototype.md
9	Verification and evaluation plan	docs/09-verification-evaluation-plan.md
10	Project management and risks	docs/10-project-management-and-risks.md
11	Supervisor feedback	docs/11-supervisor-feedback.md

A.3 Diagram sources and exports

Diagram	Editable source	PNG export
System context	01-system-context.dot	PNG
Use cases	02-use-case.dot	PNG

Diagram	Editable source	PNG export
Components (draft)	03-component.dot	PNG
Data model (ER)	04-data-model.mmd	PNG
Sequence: submission and event	05-sequence-submission-and-event.mmd	PNG

A.4 Mockups

Screen	Image
1. Log in	s1-login.png
2. Contest list	s2-contest-list.png
3. Problem, editor and timer	s3-problem-editor.png
4. Integrity warning	s4-warning.png
5. Create contest	s5-create-contest.png
6. Live monitoring	s6-monitoring.png